HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

TIỂU LUẬN 03

TRÍ TUỆ NHÂN TẠO VÀ HỌC MÁY

CNN, RNN, KNOWLEDGE GRAPH VÀ RAG

Học phần: Phát triển các hệ thống thông minh

Giảng viên hướng dẫn: Thầy Trần Đình Quế

Sinh viên thực hiện: Kiều Trường Giang

Mã sinh viên: B23DCCN254

Hà Nội - 2026

MỤC LỤC

CHƯƠNG 1. LỊCH SỬ VÀ PHÁT TRIỂN AI.....5
1.1. AI là gì? Bản chất của Trí tuệ Nhân tạo.....5
1.2. Biên niên sử phát triển AI qua các kỷ nguyên.....6
1.3. Bản đồ phân loại toàn cảnh các kỹ thuật AI.....10
1.4. Ba trụ cột tạo nên sự bùng nổ của Deep Learning sau 2012.....11
1.5. Phân loại AI theo mức độ năng lực và Trường phái tiếp cận.....11
1.5.1. Phân loại theo cấp độ năng lực.....11
1.5.2. Hai trường phái tư tưởng lớn trong AI.....12
1.6. Xu hướng AI tại Việt Nam và Những thách thức đương đại.....12
1.6.1. Bức tranh phát triển AI tại Việt Nam.....12
1.6.2. Những thách thức cốt lõi đặt ra.....12
1.7. Tiểu kết chương 1.....13
1.8. Mối liên hệ giữa các giai đoạn phát triển.....13
CHƯƠNG 2. CÁC KỸ THUẬT MACHINE LEARNING CƠ BẢN.....15
2.1. Cơ sở lý thuyết Học máy (Machine Learning Foundations).....15
2.1.1. Khái niệm và Các dạng học máy.....15
2.1.2. Quy trình chuẩn xây dựng một mô hình Học máy.....16
2.1.3. Phương pháp Tối ưu hóa bằng Hạ Gradient (Gradient Descent).....17
2.1.4. Hồi quy tuyến tính (Linear Regression).....18
2.1.5. Hồi quy Logistic (Logistic Regression & Softmax).....18
2.1.6. Tổng quan các thuật toán Học máy kinh điển tiêu biểu.....19
2.1.7. Hiện tượng Overfitting, Underfitting & Kỹ thuật Điều chuẩn.....20
2.1.8. Từ Học máy truyền thống đến Mạng Nơ-ron Sâu (Deep Neural Network – DNN).....21
2.1.9. Kiểm định chéo và lựa chọn siêu tham số.....22
2.1.10. Các thước đo đánh giá.....22
2.2. Phân tích và chuẩn bị dữ liệu thực nghiệm.....23
2.3. So sánh và lựa chọn mô hình.....24
2.4. Kết quả test độc lập và phân tích sai số.....25
2.5. Kết luận chương.....26
CHƯƠNG 3. MẠNG NƠ-RON TÍCH CHẬP CNN.....27
3.1. Lý thuyết CNN.....27
3.1.1. Tại sao cần CNN?.....27
3.1.2. Phép Tích chập (Convolution).....27
3.1.3. Pooling.....27
3.1.4. Kiến trúc CNN điển hình.....27
3.1.5. Các kiến trúc CNN tiêu biểu.....28

3.1.6. Backpropagation, loss và optimizer....28
3.1.7. Transfer learning và data augmentation....29
3.2. Dữ liệu ảnh và kiến trúc thực nghiệm....29
3.3. Kết quả và phân tích dự đoán....30
3.4. Suy luận, triển khai và phạm vi mở rộng....31
CHƯƠNG 4. MẠNG NƠ-RON HỒI QUY RNN....32
4.1. Lý thuyết RNN....32
4.1.1. Vấn đề với dữ liệu tuần tự....32
4.1.2. RNN cơ bản — Vanilla RNN....32
4.1.3. LSTM — Long Short-Term Memory....32
4.1.4. GRU — Gated Recurrent Unit....33
4.1.5. Cửa sổ trượt và các kiểu đầu vào đầu ra....33
4.1.6. BPTT và ổn định gradient....34
4.2. Dữ liệu, phép chia và cấu hình....34
4.3. So sánh RNN, LSTM, GRU và Persistence....35
4.4. Hạn chế và khả năng mở rộng....36
CHƯƠNG 5. KNOWLEDGE GRAPH VÀ RAG....37
5.1. MỞ ĐẦU....37
5.1.1. LỊCH SỬ PHÁT TRIỂN CỦA KNOWLEDGE GRAPH....37
5.1.2. LỊCH SỬ PHÁT TRIỂN CỦA RAG....38
5.1.3. VAI TRÒ VÀ Ý NGHĨA TRONG AI....38
5.1.4. PHẠM VI, CÂU HỎI VÀ ĐÓNG GÓP....39
5.2. KNOWLEDGE GRAPH - KNOWLEDGE BASE....39
5.2.1. BỘ BA, THỰC THỂ VÀ QUAN HỆ....40
5.2.2. KNOWLEDGE GRAPH CORE ENGINE....40
5.2.3. RDF VÀ LABELED PROPERTY GRAPH....41
5.2.4. CHẤT LƯỢNG VÀ QUẢN TRỊ TRI THỨC....41
5.3. RAG - RETRIEVAL-AUGMENTED GENERATION....42
5.3.1. NẠP, LẬP CHỈ MỤC VÀ TRUY XUẤT....42
5.3.2. TRUY XUẤT ĐỒ THỊ VÀ SUY LUẬN NHIỀU BƯỚC....43
5.3.3. TỔNG HỢP CÂU TRẢ LỜI CÓ CĂN CỨ....44
5.3.4. ĐÁNH GIÁ RAG VÀ GRAPHRAG....44
5.4. KB VÀ RAG CHO BỆNH TIỂU ĐƯỜNG....45
5.4.1. ONTOLOGY VÀ XỬ LÝ DỮ LIỆU Y TẾ....45
5.4.2. THỰC NGHIỆM HỎI ĐÁP Y TẾ....46
5.4.3. GIỚI HẠN VÀ HƯỚNG MỞ RỘNG Y TẾ....47
5.5. KB VÀ RAG CHO CHỨNG KHOÁN....47
5.5.1. DỮ LIỆU VÀ TIỀN XỬ LÝ FPT....48
5.5.2. HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH FPT....49

5.5.3. HỎI ĐÁP CHỨNG KHOÁN VỚI GRAPHRAG....49
5.5.4. RỦI RO VÀ HƯỚNG MỞ RỘNG....50
5.6. KB VÀ RAG CHO GIÁ VÀNG....50
5.6.1. DỮ LIỆU, CHỈ BÁO VÀ KNOWLEDGE GRAPH VÀNG....51
5.6.2. HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH VÀNG....52
5.6.3. HỎI ĐÁP VĨ MÔ VỚI GRAPHRAG....53
5.6.4. GIỚI HẠN VÀ HƯỚNG MỞ RỘNG....53
5.7. DEPLOYMENT....54
5.7.1. SUY LUẬN MÔ HÌNH TRÊN TRÌNH DUYỆT....54
5.7.2. CHATBOT DỰ ĐOÁN, TƯ VẤN VÀ HỎI ĐÁP....55
5.8. Đánh giá tổng hợp....55
5.8.1. Kết quả huấn luyện ba miền....56
5.8.2. Kết quả truy xuất và bộ ba tri thức....56
5.8.3. Chunking, embedding và giới hạn đánh giá....58
5.8.4. Đồng bộ notebook và các chức năng web bổ sung....58
5.9. Kết luận chương....58

CHƯƠNG 1. LỊCH SỬ VÀ PHÁT TRIỂN AI

1.1. AI là gì? Bản chất của Trí tuệ Nhân tạo

Trí tuệ nhân tạo (Artificial Intelligence – AI) là một phân ngành khoa học máy tính chuyên sâu, nghiên cứu và xây dựng các hệ thống tính toán có khả năng thực thi những tác vụ vốn dĩ đòi hỏi năng lực nhận thức và trí tuệ của con người: nhận thức không gian thị giác, phân tích quy nạp, suy luận logic, hiểu ngôn ngữ tự nhiên, học hỏi kinh nghiệm từ quá khứ và giải quyết vấn đề trong môi trường bất định.

Hội thảo Dartmouth năm 1956 đánh dấu sự hình thành của AI như một lĩnh vực nghiên cứu. John McCarthy dùng tên gọi Artificial Intelligence cho hướng xây dựng các hệ thống máy tính thông minh.

Trong giáo trình kinh điển thế giới Artificial Intelligence: A Modern Approach, hai tác giả Stuart Russell và Peter Norvig đã tổng hợp và phân loại các cách tiếp cận AI trong lịch sử theo hai chiều kích: Tư duy vs. Hành động và Mô phỏng con người vs. Tính duy lý (Hợp lý):

Bảng 1.1. Bốn hướng tiếp cận trí tuệ nhân tạo

Chiều kích	Tiếp cận dựa trên con người	Tiếp cận dựa trên tính hợp lý
Tư duy	Tư duy như con người (Thinking Humanly). Khoa học nhận thức; mô phỏng quá trình nơ-ron não bộ xử lý thông tin.	Tư duy duy lý (Thinking Rationally). Dựa trên các quy luật logic hình thức (Boole, Frege); rút ra kết luận đúng đắn từ các tiền đề.
Hành động	Hành động như con người (Acting Humanly). Tiêu biểu là phép thử Turing (1950); hành vi máy khiến con người tin rằng nó là người.	Hành động duy lý (Acting Rationally). Tác tử hợp lý quan sát môi trường và hành động tối đa hóa kỳ vọng đạt mục tiêu.

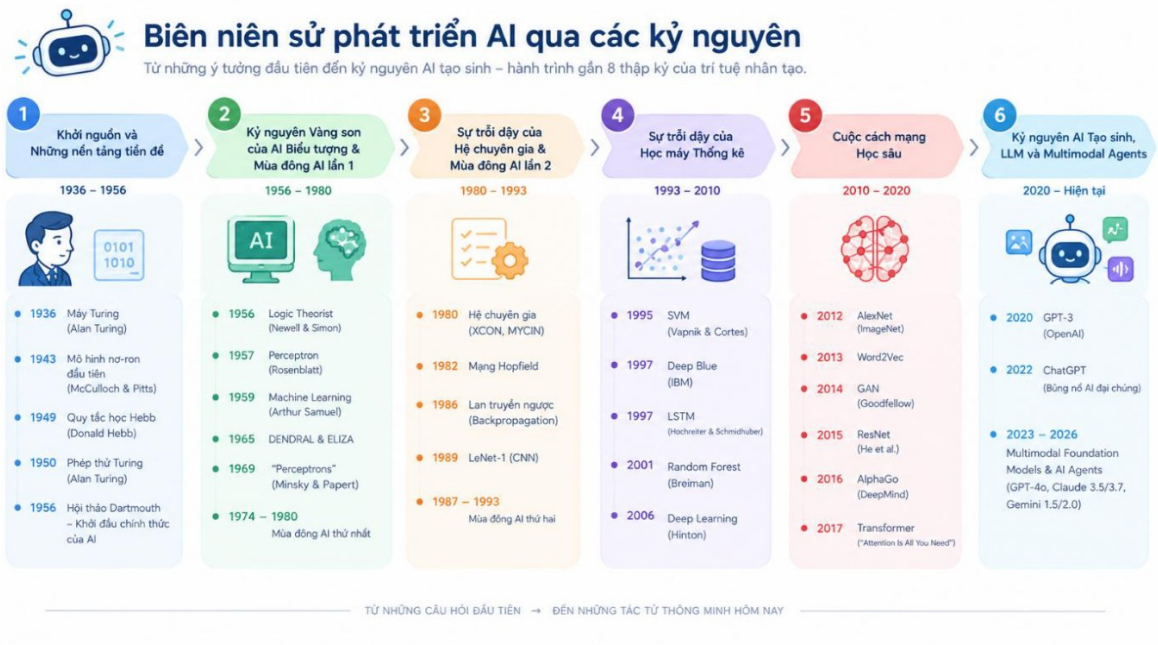
Cách tiếp cận hiện đại chủ đạo: Ngày nay, cộng đồng AI toàn cầu chọn cách tiếp cận Tác tử hợp lý (Rational Agent) làm nền tảng. Thay vì cố gắng sao chép từng hành vi cảm tính của con người, hệ thống AI được thiết kế để nhận tri giác (Percepts) thông qua cảm biến, xử lý thông tin bằng thuật toán toán học và đưa ra hành động (Actions) thông qua cơ cấu chấp hành nhằm tối đa hóa thước đo hiệu năng (Performance Measure).

Phân biệt mối quan hệ giữa các khái niệm cốt lõi:

- AI (Trí tuệ nhân tạo): Chiếc ô rộng nhất, bao gồm cả AI dựa trên luật (Rule-based), logic hình thức, hệ chuyên gia và học máy.
- Machine Learning (Học máy): Nhánh con trọng yếu của AI, tập trung vào việc tự động rút ra quy luật thống kê từ dữ liệu.
- Deep Learning (Học sâu): Nhánh con của học máy, sử dụng mạng nơ-ron nhiều lớp (Deep Neural Networks) để tự động học biểu diễn đặc trưng phân cấp từ dữ liệu thô.

1.2. Biên niên sử phát triển AI qua các kỷ nguyên

Lịch sử phát triển gần 8 thập kỷ của AI là một chuỗi hành trình kỳ vĩ, đan xen giữa những thời kỳ lạc quan tột độ và những đợt suy thoái đóng băng tài trợ ("Mùa đông AI"). Nhiều bước tiến công nghệ bắt nguồn từ sự thôi thúc vượt qua những rào cản kỹ thuật của thế hệ đi trước.



Hình 1.1. Các giai đoạn và quan hệ giữa các kỹ thuật AI

Giai đoạn 1: Khởi nguồn và Những nền tảng tiền đề (1936 – 1956)

- 1936 — Máy Turing (Alan Turing): Alan Turing công bố công trình lịch sử "On Computable Numbers", hình thức hóa khái niệm "thuật toán" và "tính toán" thông qua cỗ máy trừu tượng Turing Machine. Đây là nền móng toán học khẳng định khái niệm tính được bằng thuật toán có thể được mô hình hóa chặt chẽ; điều này không chứng minh mọi hoạt động tư duy đều tính được.

- 1943 — Mô hình nơ-ron nhân tạo đầu tiên (Warren McCulloch & Walter Pitts): Đề xuất mô hình toán học nhị phân mô phỏng tế bào thần kinh sinh học, chứng minh rằng một mạng lưới các nơ-ron logic có thể tính toán bất kỳ hàm logic nào.
- 1949 — Quy tắc học Hebb (Donald Hebb): Cuốn sách "The Organization of Behavior" chỉ ra rằng khi hai tế bào thần kinh được kích hoạt đồng thời, liên kết giữa chúng sẽ mạnh lên ("Cells that fire together, wire together"). Đây là nguồn gốc của việc điều chỉnh trọng số liên kết trong mạng nơ-ron.
- 1950 — Phép thử Turing & Câu hỏi lịch sử: Alan Turing công bố bài báo "Computing Machinery and Intelligence", đặt ra câu hỏi bất hủ: "Máy móc có thể suy nghĩ được không?" và giới thiệu phép thử Turing (Imitation Game).
- 1956 — Hội thảo Dartmouth — Điểm khởi đầu chính thức: John McCarthy, Marvin Minsky, Nathaniel Rochester và Claude Shannon đồng tổ chức hội thảo 2 tháng tại Đại học Dartmouth. Thuật ngữ "Artificial Intelligence" chính thức ra đời, đánh dấu sự hình thành của một ngành khoa học độc lập.

Giai đoạn 2: Kỷ nguyên Vàng son của AI Biểu tượng & Mùa đông AI lần 1 (1956 – 1980)

- 1956 — Logic Theorist (Newell & Simon): Chương trình AI đầu tiên có khả năng chứng minh 38 trong 52 định lý của chương 2 cuốn sách Principia Mathematica, thậm chí tìm ra cách chứng minh ngắn gọn hơn cả các tác giả gốc.
- 1957 — Perceptron (Frank Rosenblatt): Phát minh ra thuật toán Perceptron – mô hình học máy đầu tiên có thể học trọng số từ dữ liệu để phân loại nhị phân trên phần cứng Mark I Perceptron. Rosenblatt tuyên bố máy sẽ sớm có thể nhìn, nói và tự sinh sản.
- 1959 — Khái niệm "Machine Learning" (Arthur Samuel): Viết chương trình chơi cờ đam (checkers) tự cải thiện năng lực qua từng ván đấu và định nghĩa Machine Learning: "Khoa học giúp máy tính có khả năng tự học mà không cần được lập trình chi tiết từng bước".
- 1965 — DENDRAL & ELIZA (1966): Hệ thống DENDRAL phân tích cấu trúc phân tử hóa học (tiền thân của hệ chuyên gia); Joseph Weizenbaum phát triển chatbot ELIZA mô phỏng nhà tâm lý học giao tiếp qua khớp mẫu từ khóa.
- 1969 — Cuốn sách "Perceptrons" (Minsky & Papert): Cuốn sách toán học chỉ ra giới hạn chết người của Perceptron đơn lớp: hoàn toàn không thể giải quyết bài toán XOR (bài toán phân tách phi tuyến). Kết luận bi quan này khiến làn sóng nghiên cứu mạng nơ-ron bị đình trệ nghiêm trọng suốt hơn một thập kỷ.

- 1974–1980 — Mùa đông AI thứ nhất (The First AI Winter): Báo cáo Lighthill (1973) tại Anh và các đánh giá của DARPA tại Mỹ kết luận AI đã hứa hẹn quá mức so với thực tế ("Bùng nổ tổ hợp", máy tính không đủ bộ nhớ để giải bài toán ngoài "thế giới đồ chơi"). Tài trợ bị cắt giảm ồ ạt, các dự án AI bước vào giai đoạn thoái trào sâu rộng.

Giai đoạn 3: Sự trỗi dậy của Hệ chuyên gia & Mùa đông AI lần 2 (1980 – 1993)

- 1980 — Sự bùng nổ của Hệ chuyên gia (Expert Systems): AI chuyển hướng sang AI tri thức biểu tượng. Hệ thống mã hóa tri thức chuyên gia dưới dạng hàng nghìn luật "NẾU... THÌ...". Hệ thống XCON của Digital Equipment Corporation (DEC) giúp tiết kiệm 40 triệu USD/năm; MYCIN chẩn đoán nhiễm trùng máu chính xác ngang chuyên gia hàng đầu.
- 1982 — Mạng Hopfield (John Hopfield): Giới thiệu mạng nơ-ron hồi quy có khả năng lưu trữ thông tin như bộ nhớ liên kết (Associative Memory), làm bừng tỉnh lại cộng đồng nghiên cứu mạng nơ-ron.
- 1986 — Thuật toán Lan truyền ngược (Backpropagation): David Rumelhart, Geoffrey Hinton và Ronald Williams xuất bản bài báo kinh điển trên tạp chí Nature, tái khám phá và phổ biến giải thuật Lan truyền ngược lỗi cho mạng nơ- ron nhiều lớp. Giới hạn hàm XOR của Minsky chính thức bị đập tan.
- 1989 — LeNet-1 (Yann LeCun): Yann LeCun sáng tạo ra mạng nơ-ron tích chập (CNN) kết hợp lan truyền ngược, ứng dụng thành công trong việc nhận dạng chữ số viết tay trên séc ngân hàng bưu điện Mỹ.
- 1987–1993 — Mùa đông AI thứ hai (The Second AI Winter): Các hệ chuyên gia bộc lộ điểm nghẽn nghiêm trọng: chi phí bảo trì khổng lồ, "nút thắt thu thập tri thức" (Knowledge Acquisition Bottleneck), hệ thống giòn giã (brittle) – thất bại hoàn toàn khi gặp tình huống ngoại lệ. Thị trường phần cứng chuyên biệt LISP sụp đổ trước sự vươn lên của PC thương mại. AI một lần nữa rơi vào khủng hoảng lòng tin.

Giai đoạn 4: Sự trỗi dậy của Học máy Thống kê (1993 – 2010)

Từ bỏ tham vọng giải quyết trí tuệ tổng quát bằng các luật cứng nhắc, AI chuyển dịch trọng tâm sang phương pháp thống kê, xác suất và tối ưu hóa lồi:

- 1995 — Máy vector hỗ trợ (Support Vector Machine – SVM): Vladimir Vapnik và Corinna Cortes công bố thuật toán SVM với nguyên lý tối đa hóa biên phân cách (Margin Maximization) và kỹ thuật Kernel Trick. SVM nhanh chóng trở thành công cụ phân loại thống trị nhờ cơ sở lý thuyết toán học vững chắc và tránh được cực tiểu địa phương.

- 1997 — Deep Blue đánh bại Garry Kasparov: Siêu máy tính Deep Blue của IBM đánh bại nhà vô địch cờ vua thế giới Garry Kasparov, tạo cơn địa chấn toàn cầu về năng lực tính toán của máy tính.
- 1997 — Mạng LSTM ra đời (Hochreiter & Schmidhuber): Sepp Hochreiter và Jürgen Schmidhuber phát minh kiến trúc Long Short-Term Memory (LSTM), giảm khó khăn học phụ thuộc dài hạn do biến mất gradient trong mạng RNN, đặt nền móng cho xử lý chuỗi thời gian và NLP hiện đại.
- 2001 — Random Forest (Leo Breiman): Đề xuất thuật toán Rừng ngẫu nhiên kết hợp Bagging và lựa chọn ngẫu nhiên tập đặc trưng, trở thành thuật toán mạnh mẽ và bền bỉ hàng đầu cho dữ liệu dạng bảng.
- 2006 — Geoffrey Hinton và "Deep Learning": Hinton cùng các cộng sự công bố kỹ thuật tiền huấn luyện từng lớp (Layer-wise Pre-training) cho mạng Deep Belief Networks, góp phần khôi phục quan tâm đến huấn luyện mạng sâu và nhen nhóm ngọn lửa cách mạng học sâu.

Giai đoạn 5: Cuộc cách mạng Học sâu (Deep Learning Revolution) (2010 – 2020)

- 2012 — AlexNet và bước ngoặt ImageNet: Alex Krizhevsky, Ilya Sutskever và Geoffrey Hinton giành chiến thắng áp đảo tại cuộc thi ILSVRC với mạng CNN 8 lớp mang tên AlexNet, huấn luyện song song trên 2 GPU NVIDIA GTX 580. Tỉ lệ lỗi Top-5 giảm sốc từ 26% xuống 15.3%. Khoảnh khắc này đánh dấu sự thống trị tuyệt đối của Deep Learning trước các phương pháp kỹ nghệ đặc trưng thủ công.
- 2013 — Word2Vec (Tomas Mikolov / Google): Ánh xạ từ ngữ thành vector ngữ nghĩa trong không gian liên tục (Vector Embedding), nắm bắt quan hệ ngữ nghĩa tinh tế.
- 2014 — Mạng sinh đối kháng GAN (Ian Goodfellow): Đề xuất kiến trúc hai mạng Generator và Discriminator thi đấu đối kháng, khai sinh lĩnh vực tạo sinh hình ảnh chân thực.
- 2015 — Mạng ResNet (Kaiming He et al.): Sáng tạo cơ chế kết nối tắt (Residual Skip Connections), cho phép huấn luyện thành công các mạng nơ-ron sâu tới 152 lớp mà không bị suy thoái gradient, vượt qua độ chính xác của con người trên ImageNet.
- 2016 — AlphaGo đánh bại Lee Sedol: Mô hình AlphaGo (DeepMind) kết hợp Deep RL và tìm kiếm cây Monte Carlo hạ gục kỳ thủ cờ vây hàng đầu thế giới 4- 1, phá vỡ định kiến rằng cờ vây nằm ngoài khả năng của AI.

• 2017 — Kiến trúc Transformer ("Attention Is All You Need"): Đội ngũ nghiên cứu Google đề xuất kiến trúc Transformer dựa thuần túy trên cơ chế tự chú ý (Self-Attention), loại bỏ hoàn toàn cấu trúc tuần tự của RNN, tạo bước ngoặt khai sinh các mô hình nền tảng như BERT, GPT.

Giai đoạn 6: Kỷ nguyên AI Tạo sinh, LLM và Multimodal Agents (2020 – Hiện tại)

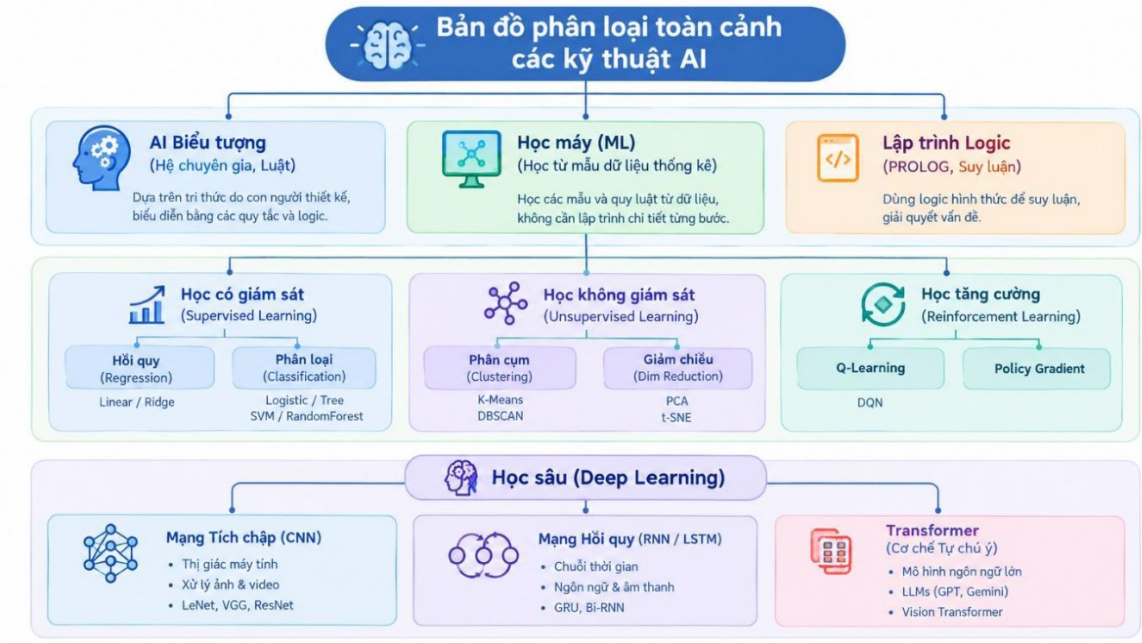
• 2020 — GPT-3 (OpenAI): Mô hình ngôn ngữ với 175 tỉ tham số thể hiện năng lực học vài ví dụ (Few-shot In-context Learning) phi thường mà không cần tái huấn luyện trọng số.

• 2022 — ChatGPT & Bùng nổ AI Đại chúng: Kết hợp tinh chỉnh chỉ dẫn (Instruction Tuning) và Học tăng cường từ phản hồi của con người (RLHF), ChatGPT cán mốc 100 triệu người dùng chỉ sau 2 tháng, mở màn cuộc cách mạng AI tạo sinh thương mại hóa toàn cầu.

• Từ 2023: Mô hình nền tảng đa phương thức kết hợp văn bản, hình ảnh và âm thanh. Các tác tử dùng mô hình ngôn ngữ để lựa chọn công cụ và thực hiện chuỗi thao tác, đồng thời cần kiểm soát quyền, đánh giá lỗi và giám sát con người.

1.3. Bản đồ phân loại toàn cảnh các kỹ thuật AI

Dưới đây là sơ đồ kiến trúc phân cấp toàn cảnh, thể hiện mối quan hệ hữu cơ giữa các trường phái Trí tuệ Nhân tạo từ cổ điển đến hiện đại:



Hình 1.2. Các giai đoạn và quan hệ giữa các kỹ thuật AI

1.4. Ba trụ cột tạo nên sự bùng nổ của Deep Learning sau 2012

Tại sao các ý tưởng mạng nơ-ron đã xuất hiện từ thập niên 1980 nhưng phải đợi tới sau năm 2012 mới tạo nên cuộc cách mạng công nghệ bùng nổ? Câu trả lời nằm ở sự hội tụ đồng thời của ba trụ cột nền tảng:

Bảng 1.2. Ba trụ cột phát triển của học sâu

Trụ cột	Bản chất kỹ thuật	Cột mốc minh họa
Dữ liệu số khổng lồ	Mạng nơ-ron sâu chứa hàng triệu đến hàng tỷ tham số, do đó đòi hỏi tập dữ liệu khổng lồ để tránh overfitting và đạt khả năng tổng quát hóa cao.	Dự án ImageNet gán nhãn hơn 14 triệu hình ảnh, cung cấp dữ liệu tiêu chuẩn cho mô hình học sâu.
Tính toán GPU	Mạng nơ-ron là chuỗi phép nhân ma trận song song; kiến trúc nhiều nhân của GPU phù hợp hơn CPU truyền thống.	CUDA xuất hiện năm 2007; AlexNet năm 2012 huấn luyện trên hai GPU GTX 580.
Thuật toán và kiến trúc	ReLU, Dropout, BatchNorm, Adam và kết nối tắt hỗ trợ tối ưu mạng sâu và hạn chế quá khớp.	Dropout (2012), BatchNorm (2015), Adam (2014), ResNet (2015).

1.5. Phân loại AI theo mức độ năng lực và Trường phái tiếp cận

1.5.1. Phân loại theo cấp độ năng lực

- AI Hẹp (Artificial Narrow AI – ANI): Hệ thống AI được tối ưu hóa xuất sắc cho một nhiệm vụ duy nhất cụ thể (chơi cờ vây, nhận dạng khuôn mặt, dịch máy, phân loại ảnh). Mặc dù thể hiện sự vượt trội so với con người trong miền hẹp, ANI hoàn toàn không có nhận thức hay khả năng tự thích ứng ngoài phạm vi được huấn luyện. Toàn bộ các hệ thống AI ứng dụng hiện nay đều thuộc cấp độ này.
- AI Tổng quát (Artificial General Intelligence – AGI): Hệ thống trí tuệ nhân tạo giả định có năng lực tư duy, suy luận, tự học và giải quyết vấn đề linh hoạt tương đương trí tuệ con người trên mọi lĩnh vực tri thức. Hiện nay, các mô hình nền tảng đa phương thức (Foundation Models) đang từng bước tiệm cận các khía cạnh nhận thức của AGI nhưng vẫn chưa đạt tới mục tiêu này.
- Siêu trí tuệ nhân tạo (Artificial Superintelligence – ASI): Khái niệm giả thuyết về một hệ thống AI sở hữu năng lực trí tuệ vượt xa những bộ não xuất sắc nhất của nhân loại ở mọi lĩnh vực khoa học, sáng tạo nghệ thuật và kỹ năng xã hội.

1.5.2. Hai trường phái tư tưởng lớn trong AI

- Trường phái Biểu tượng (Symbolic AI / Good Old-Fashioned AI – GOFAI): Tiếp cận theo hướng "từ trên xuống" (Top-down). Trí tuệ được mô hình hóa bằng các biểu tượng rõ ràng và các phép toán logic hình thức. Ưu điểm là tính minh

bạch và suy luận chặt chẽ; nhược điểm là không thể xử lý tốt sự nhiễu loạn và tính liên tục của thế giới thực.

- Trường phái Kết nối (Connectionist AI / Neural Networks): Tiếp cận theo hướng "từ dưới lên" (Bottom-up). Trí tuệ nảy sinh từ sự tương tác phức tạp của hàng triệu nơ-ron đơn giản kết nối với nhau, học hỏi trực tiếp từ mẫu dữ liệu số. Ưu điểm là khả năng thích ứng phi thường với dữ liệu phi cấu trúc; nhược điểm là tính chất "hộp đen" khó diễn giải tường minh.

1.6. Xu hướng AI tại Việt Nam và Những thách thức đương đại

1.6.1. Bức tranh phát triển AI tại Việt Nam

Việt Nam đang có những bước tiến vượt bậc trong nghiên cứu và ứng dụng trí tuệ nhân tạo:

- Khung chính sách chiến lược: Ban hành Quyết định 127/QĐ-TTg ngày 26/01/2021 về Chiến lược quốc gia về nghiên cứu, phát triển và ứng dụng AI đến năm 2030, định vị AI là công nghệ nền tảng thúc đẩy chuyển đổi số quốc gia.
- Trung tâm nghiên cứu & Doanh nghiệp tiên phong: Sự ra đời của các viện nghiên cứu đẳng cấp quốc tế như VinAI Research, Viettel AI, FPT AI Center, Zalo AI Lab... đã đóng góp nhiều công trình công bố tại các hội nghị đỉnh cao (NeurIPS, CVPR, ICML) và thương mại hóa các giải pháp trợ lý ảo tiếng Việt, xe tự hành, camera thông minh.
- Ứng dụng thực tiễn: Tích hợp AI vào hành chính công, chẩn đoán y tế từ xa, nông nghiệp công nghệ cao và phòng chống thiên tai.

1.6.2. Những thách thức cốt lõi đặt ra

1. Tính chất "Hộp đen" & Khả năng giải thích (Explainable AI – XAI): Các mạng nơ-ron sâu đưa ra dự báo chính xác nhưng con người khó truy xuất lý do tại sao, gây rủi ro lớn trong các lĩnh vực nhạy cảm như chẩn đoán ung thư, phê duyệt tín dụng hay tư pháp.

2. Thiên lệch dữ liệu & Tính công bằng (Data Bias & Fairness): Mô hình học từ dữ liệu lịch sử của con người nên có nguy cơ phản ánh và khuếch đại các định kiến xã hội về giới tính, chủng tộc hoặc địa lý.

3. Độ tin cậy & Hiện tượng "Ảo giác" (Hallucination): Các mô hình ngôn ngữ lớn có xu hướng sinh ra thông tin sai lệch nhưng với giọng điệu vô cùng tự tin, gây nguy hại cho việc tiếp nhận tri thức chính xác.

4. Chi phí tính toán & Tác động môi trường (Compute Cost & Green AI): Huấn luyện một mô hình nền tảng hiện đại tiêu tốn hàng chục triệu USD và năng lượng đáng kể, phụ thuộc quy mô và phần cứng, đặt ra bài toán về tính bền vững và sự bất bình đẳng công nghệ giữa các quốc gia.

5. Quyền riêng tư dữ liệu & An toàn bảo mật (Privacy & Adversarial Robustness): Nguy cơ rò rỉ dữ liệu cá nhân khi huấn luyện mô hình và khả năng mô hình bị đánh lừa bởi các cuộc tấn công đối nghịch (Adversarial Attacks).

1.7. Tiểu kết chương 1

Lịch sử phát triển của Trí tuệ Nhân tạo là một hành trình biện chứng đầy cảm hứng kéo dài gần 70 năm. Từ những phác thảo lý thuyết trừu tượng của Alan Turing, sự ra đời của Perceptron, qua hai cuộc "Mùa đông AI" đầy thử thách, cho đến sự phục hưng của học máy thống kê và cuộc cách mạng học sâu chấn động thế giới hôm nay.

Những chu kỳ thăng trầm lịch sử đã để lại một bài học vô giá: Không có một mô hình đơn lẻ nào là vạn năng. Để làm chủ công nghệ và xây dựng các hệ thống thông minh bền vững, người kỹ sư cần thấu hiểu bản chất toán học từ các kỹ thuật Machine Learning cổ điển, cho đến các kiến trúc chuyên biệt cho không gian (CNN) và thời gian (RNN).

Đó chính là tiền đề dẫn dắt chúng ta bước vào Chương 2, nơi các kỹ thuật học máy cơ bản và cấu trúc mạng nơ-ron sâu sẽ được mổ xẻ tường minh từ lý thuyết tới thực nghiệm.

1.8. Mối liên hệ giữa các giai đoạn phát triển

AI biểu tượng cung cấp cách biểu diễn tri thức và suy luận có thể kiểm tra. Học máy thống kê chuyển trọng tâm sang việc học quy luật từ dữ liệu; học sâu tiếp tục học biểu diễn thay cho nhiều bước thiết kế đặc trưng thủ công. Transformer cho phép mô hình hóa quan hệ giữa các vị trí qua attention và huấn luyện song song hơn RNN. KG và

RAG kết nối mô hình dữ liệu với tri thức tường minh; chúng kế thừa các ý tưởng cũ thay vì thay thế toàn bộ AI biểu tượng.

Trong báo cáo này, mối liên hệ đó được thể hiện bằng ba loại đầu vào: bảng thuộc tính, ảnh và chuỗi thời gian. Các mô hình tạo ra dự đoán định lượng; Chương 5 bổ sung bằng chứng và ngữ cảnh để người dùng hiểu phạm vi của các kết quả.

CHƯƠNG 2. CÁC KỸ THUẬT MACHINE LEARNING CƠ BẢN

Chương này giữ nền tảng học máy của Tiểu luận 01 và triển khai lại độc lập hai bài toán: phân loại nhãn nguy cơ trên Diabetes Risk Prediction 50k và hồi quy giá nhà Việt Nam. Các mô hình, phép chia dữ liệu, checkpoint và kết quả dưới đây được tạo trong main.ipynb.

2.1. Cơ sở lý thuyết Học máy (Machine Learning Foundations)

2.1.1. Khái niệm và Các dạng học máy

Năm 1997, Tom Mitchell đã đưa ra định nghĩa toán học chuẩn tắc về học máy:

"Một chương trình máy tính được coi là học từ kinh nghiệm E đối với một lớp tác vụ T và thước đo hiệu năng P, nếu hiệu năng của nó đối với các tác vụ trong T, được đo bằng P, cải thiện theo kinh nghiệm E."

Dựa vào bản chất của kinh nghiệm E (dữ liệu đầu vào), học máy được chia thành 4 dạng thức chính:

1. Học có giám sát (Supervised Learning):

• Dữ liệu huấn luyện bao gồm cặp mẫu đầu vào và nhãn mục tiêu tương ứng:

$$D=\{(x_i,y_i)\}_{i=1}^{N} \quad (2.1)$$

• Mục tiêu: Tìm hàm ánh xạ từ không gian đầu vào sang không gian đầu ra sao cho dự báo xấp xỉ nhãn thực tế.

$$f:X\to Y,\hat{y}=f(x) \quad (2.2)$$

• Hồi quy (Regression): Khi nhãn y thuộc tập số thực, đầu ra là giá trị liên tục (ví dụ: dự báo giá nhà, nhiệt độ, doanh thu).

• Phân loại (Classification): Khi nhãn là các nhãn lớp rời rạc (ví dụ: chẩn đoán bệnh tật, phân loại email rác).

2. Học không giám sát (Unsupervised Learning):

• Dữ liệu huấn luyện chỉ bao gồm các mẫu đầu vào mà không có nhãn đích:

$$D=\{x_i\}_{i=1}^{N} \quad (2.3)$$

• Mục tiêu: Tự động khám phá cấu trúc ẩn sâu bên trong dữ liệu.

• Phân cụm (Clustering): Gom các điểm dữ liệu có đặc trưng tương đồng vào cùng nhóm (K-Means, DBSCAN).

• Giảm chiều dữ liệu (Dimensionality Reduction): Nén không gian dữ liệu nhiều chiều xuống không gian ít chiều hơn mà vẫn bảo toàn tối đa thông tin quan trọng (PCA, t-SNE).

3. Học bán giám sát & Tự giám sát (Semi-supervised & Self-supervised Learning):

• Học bán giám sát: Kết hợp một lượng nhỏ dữ liệu có gán nhãn với một khối lượng khổng lồ dữ liệu không gán nhãn để tiết kiệm chi phí dán nhãn thủ công.

• Học tự giám sát: Mô hình tự động tạo ra nhãn từ chính cấu trúc nội tại của dữ liệu (ví dụ: che giấu một từ trong câu và bắt mô hình dự đoán từ bị che, tạo tiền đề cho các mô hình ngôn ngữ lớn LLM).

4. Học tăng cường (Reinforcement Learning – RL):

• Tác tử (Agent) tương tác liên tục với môi trường (Environment) qua các trạng thái st, thực hiện hành động at và nhận lại tín hiệu phần thưởng rt

(Reward). Mục tiêu là tìm chính sách hành động $\pi(a\backslash s)$ tối đa hóa tổng phần thưởng tích lũy dài hạn.

2.1.2. Quy trình chuẩn xây dựng một mô hình Học máy

Một quy trình khoa học dữ liệu hoàn chỉnh bao gồm 5 giai đoạn liên hoàn khép kín:

[1. Thu thập dữ liệu] → [2. Tiền xử lý (EDA, Clean, Scale)] → [3. Phân chia Train/Val/Test] → [4. Huấn luyện & Tối ưu Siêu tham số] → [5. Đánh giá & Giám sát]

1. Thu thập và Khám phá Dữ liệu (EDA): Phân tích phân phối thống kê, phát hiện ngoại lai (Outliers), kiểm tra độ tương quan giữa các biến và nhận diện hiện tượng rò rỉ dữ liệu (Data Leakage).

2. Tiền xử lý Dữ liệu:

• Xử lý dữ liệu khuyết: Điền khuyết (Imputation) bằng giá trị trung bình (mean), trung vị (median) hoặc mô hình học máy.

• Mã hóa biến phân loại: One-Hot Encoding cho biến định danh không thứ bậc, Label Encoding cho biến có thứ bậc.

• Chuẩn hóa thang đo: Đưa các thuộc tính về cùng một miền giá trị bằng Standard Scaling hoặc Min-Max Scaling nhằm giúp các thuật toán hạ gradient hội tụ nhanh và ổn định.

$$z = \frac{x - \mu}{\sigma}, x' = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (2.4)$$

x là giá trị gốc; μ, σ, x_min và x_max lần lượt là trung bình, độ lệch chuẩn và hai biên của đặc trưng. Các đại lượng này chỉ được ước lượng từ train, rồi giữ cố định khi biến đổi validation/test.

3. Phân chia tập dữ liệu: Chia tách độc lập thành tập Huấn luyện (Train set), tập Kiểm định (Validation set) và tập Kiểm thử (Test set). Áp dụng kỹ thuật Kiểm định chéo K-lần (K-Fold Cross-Validation) để đánh giá độ ổn định của mô hình.

4. Huấn luyện và Tối ưu hóa: Lựa chọn kiến trúc thuật toán phù hợp, định nghĩa hàm mục tiêu (Loss function) và tinh chỉnh siêu tham số (Hyperparameter Tuning).

5. Đánh giá và Kiểm định độc lập: Đo lường hiệu năng trên tập Test chưa từng nhìn thấy bằng các thước đo chuẩn mực.

2.1.3. Phương pháp Tối ưu hóa bằng Hạ Gradient (Gradient Descent)

Hạ gradient là thuật toán tối ưu hóa nền tảng trong toàn bộ học máy và học sâu, nhằm tìm bộ tham số w làm cực tiểu hóa hàm mất mát L(w).

Nguyên lý toán học: Vector gradient ∇L(w) chỉ hướng tăng nhanh nhất của hàm số tại điểm khảo sát. Do đó, để làm giảm hàm mất mát nhanh nhất, ta cập nhật tham số ngược hướng với vector gradient:

$$w_{t+1} = w_t - \eta \nabla L(w_t) \quad (2.5)$$

Trong đó η> 0 là tốc độ học (Learning Rate). Nếu quá nhỏ, mô hình hội tụ cực kỳ chậm chạp; nếu η quá lớn, mô hình sẽ dao động phân kỳ và không thể đạt tới điểm cực tiểu.

Các biến thể của Gradient Descent:

- Batch Gradient Descent: Tính toán gradient trên toàn bộ mẫu dữ liệu của tập huấn luyện. Hướng đi rất ổn định nhưng cực kỳ tốn bộ nhớ và chậm chạp khi dữ liệu lớn.
- Stochastic Gradient Descent (SGD): Tính toán gradient và cập nhật tham số trên từng mẫu đơn lẻ (N = 1). Cực kỳ nhanh và có khả năng thoát khỏi các điểm cực tiểu địa phương cạn, nhưng dao động gradient rất mạnh (nhiễu).
- Mini-batch Gradient Descent: Cân bằng hoàn hảo giữa hai thái cực trên bằng cách chia dữ liệu thành từng nhóm nhỏ kích thước B (thường là 32, 64, 128 mẫu). Đây là tiêu chuẩn vàng trong huấn luyện mạng nơ-ron hiện đại.

2.1.4. Hồi quy tuyến tính (Linear Regression)

Mô hình hồi quy tuyến tính giả thiết mối quan hệ giữa biến đầu vào x= [x1, x2,..., xd]T và biến mục tiêu liên tục y là tuyến tính:

$$\hat{y} = w^T x + b = \sum_{j=1}^{d} w_j x_j + b \quad (2.6)$$

Hàm mất mát — Sai số bình phương trung bình (Mean Squared Error – MSE):

$$L = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2 \quad (2.7)$$

Đạo hàm và Cập nhật trọng số: Đạo hàm riêng theo trọng số w:

$$\frac{\partial L}{\partial w} = \frac{2}{N} X^T (\hat{y} - y) \quad (2.8)$$

Cập nhật tham số qua gradient descent:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w} \quad (2.9)$$

2.1.5. Hồi quy Logistic (Logistic Regression & Softmax)

Dành cho bài toán phân loại. Dù mang tên "hồi quy", mô hình này thực chất là một bộ phân loại tuyến tính xác suất.

Phân loại nhị phân (Binary Classification): Để ép giá trị đầu ra tuyến tính z= wTx+ b về khoảng xác suất, mô hình sử dụng hàm kích hoạt Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2.10)$$

Hàm mất mát — Binary Cross-Entropy (BCE / Log Loss):

$$L_{BCE} = -\frac{1}{N} \sum_{i=1}^{N} [\, y_i \log \hat{y}_i + (1 - y_i) \log (1 - \hat{y}_i)] \quad (2.11)$$

Phân loại đa lớp (Multiclass Classification — Softmax Regression): Khi có K lớp phân loại rời rạc, đầu ra tại lớp thứ k được chuẩn hóa qua hàm Softmax:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \quad (2.12)$$

Hàm mất mát tương ứng là Categorical Cross-Entropy:

$$L_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\log p_{ik} \quad (2.13)$$

2.1.6. Tổng quan các thuật toán Học máy kinh điển tiêu biểu

Bên cạnh các mô hình tuyến tính, học máy cổ điển sở hữu các thuật toán phi tuyến xuất sắc:

1. Cây quyết định (Decision Tree):

- Phân chia không gian dữ liệu bằng các câu hỏi điều kiện phân nhánh nhị phân trực quan.
- Tiêu chuẩn phân chia tối ưu: sử dụng độ lợi thông tin (Information Gain) dựa trên Entropy hoặc chỉ số Gini Impurity.

$$H(S) = -\sum_{k=1}^{K} p_k \log_2 p_k, G(S) = 1 - \sum_{k=1}^{K} p_k^2 \quad (2.14)$$

pk là tỷ lệ mẫu thuộc lớp k ở nút đang xét. Độ lợi thông tin là độ giảm entropy có trọng số sau khi chia; ký hiệu p1 trong tổng của bản nguồn đã được sửa thành pk.

- Ưu điểm: Cực kỳ dễ giải thích, không yêu cầu chuẩn hóa thang đo; Nhược điểm: Dễ bị quá khớp (Overfitting) nếu cây phát triển quá sâu.

2. Phương pháp Tổ hợp (Ensemble Methods):

- Rừng ngẫu nhiên (Random Forest): Kỹ thuật đóng gói (Bagging) kết hợp dự báo của hàng trăm cây quyết định độc lập được huấn luyện trên các tập mẫu con lấy mẫu có hoàn lại (Bootstrap) và tập thuộc tính ngẫu nhiên. Triệt tiêu phương sai, cực kỳ bền bỉ và chính xác trên dữ liệu dạng bảng.
- Gradient Boosting (GBM / XGBoost / LightGBM): Kỹ thuật tăng cường (Boosting) xây dựng các cây quyết định tuần tự, trong đó mỗi cây sau tập trung sửa chữa sai số (Residuals) của các cây trước đó. Thường đạt độ chính xác cao nhất trong các cuộc thi dữ liệu bảng.

3. Máy vector hỗ trợ (Support Vector Machine – SVM):

- Tìm kiếm siêu phẳng phân cách tối ưu sao cho khoảng cách (lề - margin) từ siêu phẳng tới các điểm dữ liệu gần nhất của hai lớp (gọi là các Support Vectors) là cực đại.
- Kernel Trick: Ánh xạ dữ liệu không phân tách tuyến tính từ không gian gốc lên không gian nhiều chiều hơn thông qua các hàm nhân (RBF Kernel, Polynomial Kernel) để phân tách tuyến tính mà không cần tính toán tọa độ rõ ràng.

4. K láng giềng gần nhất (K-Nearest Neighbors – KNN):

- Giải thuật học lười biếng (Lazy Learning): Không có pha huấn luyện tường minh. Khi dự báo mẫu mới, thuật toán tính khoảng cách (Euclidean, Manhattan) tới toàn bộ các điểm trong tập dữ liệu và lấy biểu quyết đa số từ K điểm láng giềng gần nhất.

Bảng 2.1. So sánh các thuật toán học máy kinh điển

Thuật toán	Bản chất	Độ phức tạp	Giải thích	Ưu điểm	Hạn chế
Logistic Regression	Tuyến tính + Sigmoid	Dự đoán O(d)	Hệ số trọng số	Đơn giản, nhanh; xuất xác suất	Biên phân cách tuyến tính
Decision Tree	Cắt lát không gian	Phụ thuộc độ sâu	Cây luật NẾU–THÌ	Trực quan; ít cần scaling	Dễ quá khớp, nhạy nhiễu
Random Forest	Bagging nhiều cây	Phụ thuộc số cây, N, d và độ sâu	Feature importance	Bền bỉ, giảm phương sai	Mô hình lớn hơn, dự báo chậm hơn
Gradient Boosting	Boosting sửa sai số	Phụ thuộc số vòng và độ sâu	Trung bình	Mạnh với dữ liệu bảng	Nhạy siêu tham số, tốn thời gian
SVM	Cực đại hóa margin	Phụ thuộc kernel và số mẫu	Thấp với kernel	Mạnh với ít mẫu, nhiều chiều	Chậm trên tập lớn
KNN	Khoảng cách mẫu	Brute-force O(Nd) mỗi truy vấn	Láng giềng	Trực quan, không giả định phân phối	Tốn bộ nhớ, dự báo chậm

2.1.7. Hiện tượng Overfitting, Underfitting & Kỹ thuật Điều chuẩn

Sự đánh đổi giữa Độ chệch và Phương sai (Bias-Variance Trade-off): Tổng sai số kỳ vọng của một mô hình học máy được phân rã thành 3 thành phần:

$$E[(y-\hat{f}(x))^2] = Bias[\hat{f}(x)]^2 + Var[\hat{f}(x)] + \sigma^2 \quad (2.15)$$

- Thiếu khớp (Underfitting - High Bias): Mô hình quá đơn giản, không học đủ quy luật từ dữ liệu, sai số trên cả tập Train và Test đều cao.
- Quá khớp (Overfitting - High Variance): Mô hình quá phức tạp, ghi nhớ cả các nhiễu ngẫu nhiên của tập Train, dẫn tới sai số tập Train cực thấp nhưng sai số tập Test vọt lên rất cao.

Các kỹ thuật Điều chuẩn (Regularization): Điều chuẩn là kỹ thuật thêm một số hạng phạt vào hàm mất mát nhằm hạn chế độ lớn của các trọng số, ép mô hình đơn giản hơn:

- Điều chuẩn L2 (Ridge Regression / Weight Decay): Phạt tổng bình phương trọng số. Thu nhỏ trọng số về gần 0 nhưng không triệt tiêu hoàn toàn.

$$L_{reg} = L + \lambda \sum_i w_i^2 \quad (2.16)$$

- Điều chuẩn L1 (Lasso Regression): Phạt tổng giá trị tuyệt đối của trọng số. Có khả năng triệt tiêu trọng số về chính xác bằng 0, đóng vai trò chọn lọc đặc trưng tự động.

$$L_{reg} = L + \lambda \sum_i |w_i| \quad (2.17)$$

- Trong Mạng Nơ-ron: Áp dụng kỹ thuật Dropout (ngẫu nhiên ngắt kết nối một tỉ lệ nơ-ron trong quá trình train để chống đồng thích nghi) và Early Stopping (dừng huấn luyện khi sai số trên tập Validation bắt đầu tăng liên tục).

2.1.8. Từ Học máy truyền thống đến Mạng Nơ-ron Sâu (Deep Neural Network – DNN)

Khi dữ liệu có độ phức tạp cao và phi tuyến mạnh mẽ, việc kết hợp các tầng biến đổi tuyến tính xen kẽ các hàm kích hoạt phi tuyến dẫn tới kiến trúc Mạng nơ-ron sâu (DNN).

Có thể minh họa lý thuyết bằng một kiến trúc Mạng nơ-ron sâu 5 lớp (gồm 4 lớp ẩn Dense liên kết đầy đủ + 1 lớp đầu ra):

Input (X) ⟶ [Dense 1 + ReLU] ⟶ [Dense 2 + ReLU] ⟶ [Dense 3 + ReLU] ⟶ [Dense 4 + ReLU] ⟶ Output Layer (ŷ)

1. Lan truyền xuôi (Forward Propagation): Tại mỗi tầng ẩn $l \in \{1,2,3,4\}$:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, a^{(l)} = ReLU\left(z^{(l)}\right) \quad (2.18)$$

Tại tầng đầu ra $l = 5$:

- Với bài toán hồi quy (Housing Price): $\hat{y} = z(5)$(kích hoạt tuyến tính).
- Với bài toán phân loại đa lớp: $\hat{y} = \text{Softmax}(z(5))$.

2. Lan truyền ngược (Backpropagation theo Chain Rule): Để cập nhật ma trận trọng số, ta tính toán gradient sai số từ tầng cuối cùng lội ngược về tầng đầu theo quy tắc chuỗi giải tích:

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} \left(a^{(l-1)}\right)^T \quad (2.19)$$

$$\delta^{(l)} = \left(\left(W^{(l+1)}\right)^T \delta^{(l+1)}\right) \odot ReLU'\left(z^{(l)}\right) \quad (2.20)$$

Trong đó δ là sai số lan truyền ngược. Quy tắc chuỗi cho phép truyền gradient qua các lớp; phần thực nghiệm mới sử dụng các mô hình được mô tả riêng bên dưới.

2.1.9. Kiểm định chéo và lựa chọn siêu tham số

Train dùng để ước lượng tham số; validation dùng để kiểm tra các lựa chọn phát triển và checkpoint; test chỉ dùng cho đánh giá cuối. Trong phân loại, StratifiedKFold giữ tỷ lệ lớp trong từng fold. Với giá nhà, GroupKFold giữ các bản ghi cùng địa chỉ trong một fold. Toàn bộ imputation, scaling và one-hot encoding được fit bên trong từng fold, không fit trên toàn bộ dữ liệu trước khi chia.

Grid search so sánh tập siêu tham số hữu hạn đã định trước. Dùng macro-F1 khi các lớp mất cân bằng và MAE theo tỷ VND khi dự báo giá nhà. Không lựa chọn mô hình bằng chỉ số test. Khoảng cách train–validation–test giúp phát hiện quá khớp; một lần chia và một random seed chưa mô tả hết độ bất định.

2.1.10. Các thước đo đánh giá

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|, MSE = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \quad (2.21)$$

$$RMSE = \sqrt{MSE}, R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \acute{y})^2} \quad (2.22)$$

Trong (2.21)–(2.22), n là số mẫu, yi là giá trị thực, ŷi là dự đoán và ȳ là trung bình của nhãn trong tập đánh giá. MAE và RMSE có cùng đơn vị với nhãn, MSE có đơn vị bình phương. R2 so với dự đoán hằng bằng trung bình, có thể âm và không phải phần trăm chính xác.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}, P = \frac{TP}{TP + FP}, R = \frac{TP}{TP + FN} \quad (2.23)$$

$$F_1 = \frac{2PR}{P + R}, F_{1,macro} = \frac{1}{K}\sum_{k=1}^{K} F_{1,k} \quad (2.24)$$

TP, TN, FP và FN lần lượt là số dự đoán dương đúng, âm đúng, dương sai và âm sai trong cách xét một lớp so với các lớp còn lại. Với K lớp, macro-F1 cho mỗi lớp trọng số bằng nhau, vì vậy thể hiện rõ hơn hiệu năng ở lớp ít mẫu.

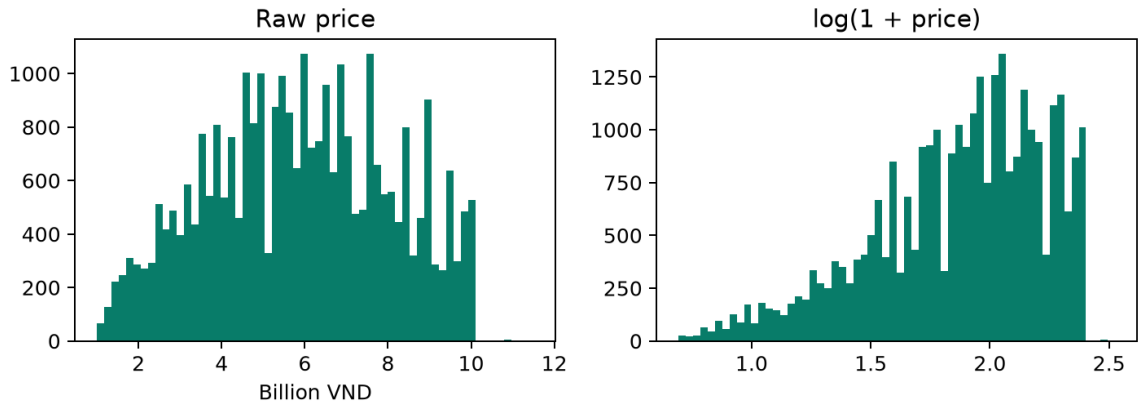
2.2. Phân tích và chuẩn bị dữ liệu thực nghiệm

Bảng 2.2. Dữ liệu của các thực nghiệm tự xây dựng lại

Dataset	Quy mô	Mục tiêu	Train / Val / Test
Diabetes Risk Prediction	50.000 dòng; 41 cột gồm cả nhãn	Ba nhãn High, Low, Moderate	32000 / 8000 / 10000
Vietnam Housing 2024	30.229 dòng; 12 cột gốc	Giá nhà theo tỷ VND	19377 / 4491 / 6361

Diabetes Risk Prediction là dữ liệu tổng hợp phục vụ học máy, không được coi là 50.000 hồ sơ bệnh nhân đã xác minh lâm sàng. Phân phối thực tế gồm High 36.593 mẫu (73,2%), Moderate 12.937 mẫu (25,9%) và Low 470 mẫu (0,9%). Vì vậy nhận định cân bằng của bản nguồn được thay bằng phân tích mất cân bằng. Tám đầu vào gồm Age, BMI, HbA1c, Fasting_Blood_Sugar, Blood_Glucose, Blood_Pressure_Systolic, HDL và LDL. Patient_ID, Diabetes_Risk_Score, AI_Health_Recommendation và Doctor_Consultation_Needed không được sử dụng vì là định danh hoặc có thể tiết lộ nhãn.

Dữ liệu giá nhà có các trường Area, Frontage, Access Road, Floors, Bedrooms, Bathrooms cùng hướng nhà, hướng ban công, pháp lý và nội thất. Province được trích từ phần cuối địa chỉ. Pipeline điền thiếu số bằng trung vị train, mã hóa one-hot với nhóm chưa gặp được bỏ qua, và học log(1 + Price) rồi biến đổi ngược trước khi tính sai số. Các bản ghi cùng địa chỉ được giữ trong cùng một tập. Đây là kiểm tra tổng quát hóa sang địa chỉ khác, chưa phải backtest giá nhà theo thời gian.



Hình 2.1. Phân phối giá nhà trước và sau biến đổi logarit

2.3. So sánh và lựa chọn mô hình

Bảng 2.3. So sánh phân loại với StratifiedKFold 3 phần

Mô hình	CV macro-F1
Majority	0.2817
Logistic Regression	0.5027
KNN	0.5055
Decision Tree	0.5610
Random Forest	0.6203
SVM RBF	0.5729

Random Forest được chọn bằng CV macro-F1 0.6203, gồm 80 cây, max_depth=10, class_weight=balanced, random_state=42. Macro-F1 train 0.7105, validation 0.6059 và test 0.6087 cho thấy vẫn có khoảng cách khái quát hóa. Không dùng test để tăng độ sâu cây.

Bảng 2.4. So sánh hồi quy với GroupKFold 3 phần

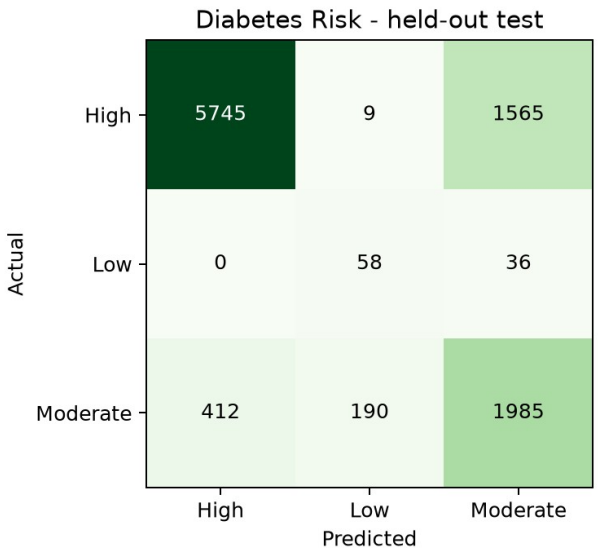
Mô hình	CV MAE (tỷ VND)
Median	1.8623
Log Ridge	1.4915
Log Random Forest	1.3091
Log Gradient Boosting	1.3246

Hồi quy chọn Log Random Forest với 80 cây, max_depth=14, min_samples_leaf=3. Ridge thử alpha 1, 10, 100; Gradient Boosting thử learning_rate 0,04 và 0,08. Tất cả cấu hình cùng một nhóm train và các fold theo địa chỉ. Mô hình trung vị cung cấp mốc đánh giá độ hữu ích của các mô hình học được.

2.4. Kết quả test độc lập và phân tích sai số

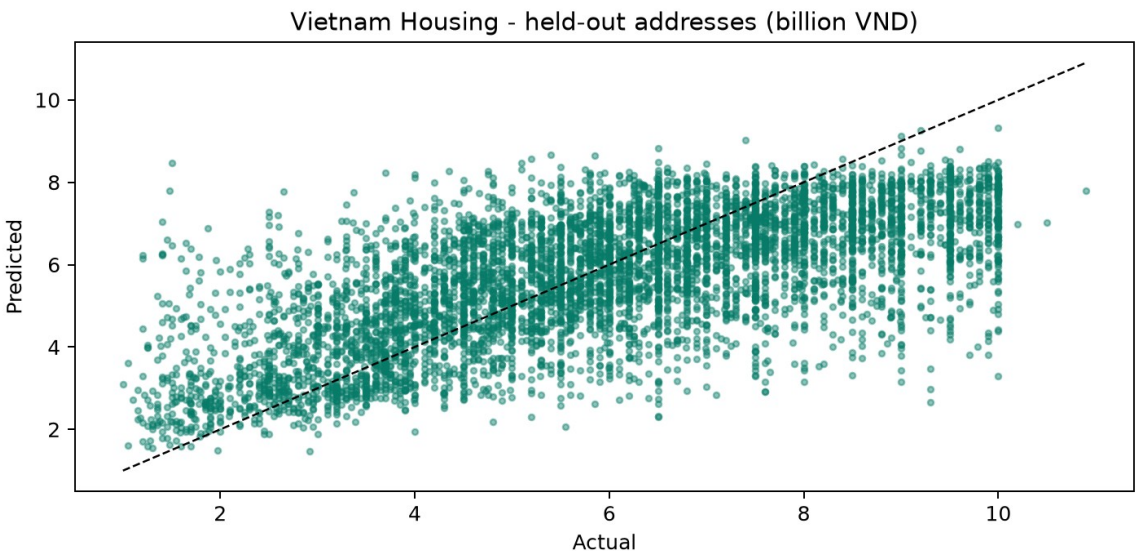
Bảng 2.5. Kết quả được tính từ predictions của main.ipynb

Bài toán	Chỉ số	Kết quả
Phân loại nguy cơ	Accuracy / Precision macro	0.7788 / 0.5708
Phân loại nguy cơ	Recall macro / F1 macro	0.7231 / 0.6087
Giá nhà	MAE / RMSE (tỷ VND)	1.2760 / 1.6394
Giá nhà	MSE / R2	2.6877 / 0.4273



Hình 2.2. Ma trận nhầm lẫn của phân loại nguy cơ trên 10.000 mẫu test

Ma trận nhầm lẫn cho thấy Accuracy chịu ảnh hưởng lớn của lớp High. Macro-precision thấp hơn macro-recall phản ánh chi phí dự đoán nhầm khi dùng class_weight=balanced. Nhãn dự đoán chỉ mô tả quy tắc của dataset, không phải chẩn đoán hoặc nguy cơ bệnh ngoài đời.



Hình 2.3. Giá nhà dự đoán và thực tế tại các địa chỉ giữ riêng

MAE train của giá nhà là 1.0276 tỷ VND, validation 1.2936, test 1.2760; R2 test 0.4273. Sai số tuyệt đối còn lớn đối với định giá cá nhân. Các thông tin vị trí trong địa chỉ, chất lượng tin đăng và thời điểm giao dịch chưa được mô hình hóa đầy đủ. Mức lệch giữa train và test cần được theo dõi trước khi tăng độ phức tạp.

2.5. Kết luận chương

Với dữ liệu dạng bảng, Random Forest và Gradient Boosting thường cạnh tranh tốt với Deep Learning. Không có một mô hình đơn lẻ phù hợp với mọi tập dữ liệu.

Hai thực nghiệm mới thay cho các kết quả Scratch/Keras/PyTorch của tác giả nguồn. Lý thuyết DNN được giữ để giải thích nền tảng, nhưng không trình bày checkpoint hoặc độ chính xác DNN cũ như sản phẩm thực nghiệm của sinh viên. Các tệp joblib, CSV dự đoán và bảng CV là bằng chứng của kết quả ở chương này.

CHƯƠNG 3. MẠNG NƠ-RON TÍCH CHẬP CNN

3.1. Lý thuyết CNN

3.1.1. Tại sao cần CNN?

Với ảnh 224×224×3 (RGB), nếu dùng Dense layer thông thường:

• Số tham số đầu vào: 224 × 224 × 3 = 150,528 giá trị

• Một Dense layer với 1000 nơ-ron: 150 triệu tham số — bùng nổ tham số!

CNN giải quyết bằng 3 ý tưởng chính:

1. Local connectivity: Mỗi nơ-ron chỉ nhìn một vùng nhỏ của ảnh

2. Weight sharing: Cùng một kernel dùng chung cho toàn bộ ảnh

3. Translation equivariance: phép tích chập dùng chung kernel khi dịch đầu vào; pooling tạo bất biến cục bộ ở mức độ nhất định.

3.1.2. Phép Tích chập (Convolution)

Với kernel K kích thước (kH × kW) và ảnh đầu vào X kích thước (H × W):

$$Z(i,j) = \sum_{u=0}^{kH-1} \sum_{v=0}^{kW-1} X(i+u, j+v) \cdot K(u,v) + b \tag{3.1}$$

Kích thước đầu ra (với padding P, stride S):

$$H_{out} = \left\lfloor \frac{H - kH + 2P}{S} \right\rfloor + 1 \tag{3.2}$$

3.1.3. Pooling

Max Pooling: Lấy giá trị lớn nhất trong mỗi vùng → giảm kích thước, giữ đặc trưng nổi bật

$$P_{max}(i,j) = \max_{(u,v) \in R_{ij}} Z(u,v) \tag{3.3}$$

Average Pooling: Lấy trung bình → giảm kích thước mượt hơn

3.1.4. Kiến trúc CNN điển hình

Input → [Conv → BN → ReLU → MaxPool] × N → Flatten → Dense → Softmax

Bảng 3.1. Vai trò của các tầng CNN

Tầng	Vai trò
Conv2D	Trích xuất đặc trưng cục bộ: cạnh, góc, texture
BatchNorm	Chuẩn hóa giúp huấn luyện ổn định hơn
ReLU	Tạo tính phi tuyến
MaxPool	Giảm kích thước, tăng receptive field
Dropout	Điều chuẩn, hạn chế quá khớp
Dense và Softmax	Phân loại đa lớp

3.1.5. Các kiến trúc CNN tiêu biểu

Bảng 3.2. Các kiến trúc CNN và đặc trưng thiết kế đã hiệu chỉnh

Kiến trúc	Năm	Đặc điểm
LeNet-5	1998	Nhận dạng chữ số, convolution và subsampling
AlexNet	2012	ReLU, Dropout và huấn luyện GPU
VGG	2014	Các kernel 3 × 3 xếp chồng
GoogLeNet	2014	Các nhánh Inception
ResNet	2015	Kết nối residual
EfficientNet	2019	Compound scaling độ sâu, độ rộng và độ phân giải

LeNet-5 không phải mô hình ImageNet năm 1989. Các con số Top-5 của nhiều biến thể kiến trúc, tập dữ liệu hoặc chế độ ensemble không được đặt cạnh nhau như một phép so sánh thống nhất. Bảng 3.2 chỉ so sánh đặc điểm kiến trúc; các chỉ số định lượng trong chương được tính riêng trên MNIST.

3.1.6. Backpropagation, loss và optimizer

Lan truyền xuôi tạo logits của từng lớp; CrossEntropyLoss kết hợp log-softmax và negative log likelihood, vì vậy không áp softmax trước loss trong PyTorch. Backpropagation dùng quy tắc chuỗi để tính gradient của kernel và dense layer. Adam

cập nhật trọng số dựa trên các ước lượng moment của gradient; learning rate, weight decay và checkpoint được lựa chọn mà không nhìn test.

$$\frac{\partial L}{\partial K_{uv}} = \sum_{i,j} \frac{\partial L}{\partial Y_{ij}} X_{i+u,j+v} \quad (3.4)$$

Trong (3.4), K là kernel, X là đầu vào và Y là đầu ra trước kích hoạt; chỉ số u,v xác định phần tử kernel và i,j chạy qua vị trí đầu ra. Cùng một trọng số được dùng tại nhiều vị trí nên gradient là tổng đóng góp ở các vị trí đó.

3.1.7. Transfer learning và data augmentation

Transfer learning khởi tạo từ mạng đã học trên tập lớn, thay đầu phân loại rồi đóng băng hoặc fine-tune một số tầng. Nó hữu ích khi dữ liệu đích ít nhưng phải công bố phần pretrained và phần được học lại. Thực nghiệm MNIST dưới đây không dùng pretrained; augmentation chỉ dịch ảnh train ±1 pixel, giữ nguyên validation và test. Không dùng phép xoay mạnh làm thay đổi ý nghĩa giữa các chữ số.

3.2. Dữ liệu ảnh và kiến trúc thực nghiệm

MNIST có 60.000 ảnh train gốc và 10.000 ảnh test độc lập, ảnh xám 28 × 28, nhãn 0–9. Tách 12.000 ảnh từ train làm validation theo stratification, còn 48.000 ảnh huấn luyện. Mỗi pixel được chia 255; không học thống kê từ test. Bộ tải đọc trực tiếp các tệp IDX nén gzip, kiểm tra magic number, số lượng ảnh và kích thước.

Bảng 3.3. Kiến trúc CNN tự huấn luyện

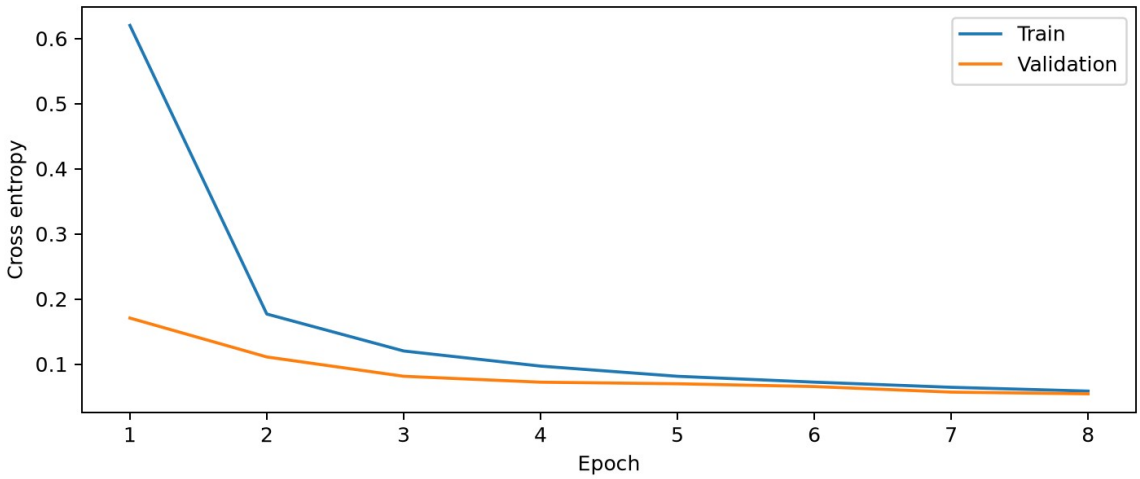
Tầng	Kích thước đầu ra	Tham số
Ảnh vào	1 × 28 × 28	Pixel / 255
Conv1 + ReLU + MaxPool	8 × 14 × 14	Kernel 3; padding 1; pool 2
Conv2 + ReLU + MaxPool	16 × 7 × 7	Kernel 3; padding 1; pool 2
Flatten + Dense + ReLU	64	784 → 64
Dense logits	10	64 → 10

Mạng có 52,138 tham số, train từ đầu 8 epoch trên CPU, Adam learning_rate=0,001, batch_size=128, weight_decay=0,0001 và gradient clipping norm=1. Checkpoint có validation loss nhỏ nhất nằm ở epoch 8. Thời gian thực nghiệm ghi trong metadata là 93.5 giây trên môi trường hiện tại.

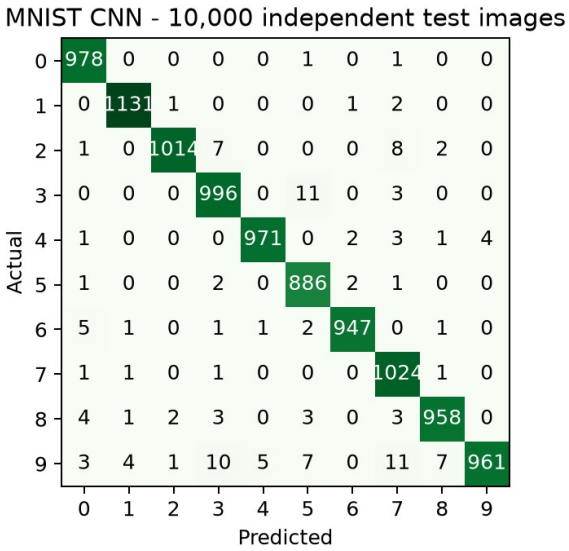
3.3. Kết quả và phân tích dự đoán

Bảng 3.4. Đánh giá CNN trên các tập độc lập

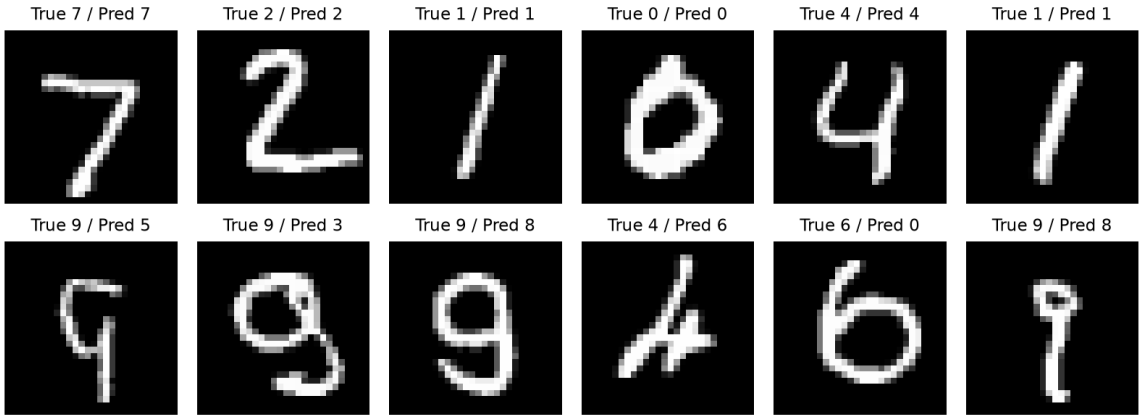
Tập	Accuracy	Macro-F1
Validation	0.9849	0.9848
Test 10.000 ảnh	0.9866	0.9865



Hình 3.1. Learning curve của CNN qua 8 epoch



Hình 3.2. Ma trận nhầm lẫn CNN trên tập test MNIST



Hình 3.3. Sáu dự đoán đầu và sáu lỗi đầu tiên theo thứ tự test

Accuracy test là 98.66%, tương đương 134 lỗi trên 10.000 ảnh. Các ảnh sai trong Hình 3.3 được chọn bằng quy tắc cố định, không chọn riêng ví dụ đẹp. Learning curve cho thấy validation loss giảm trong cấu hình đã chạy; không kết luận mọi CNN hoặc mọi dữ liệu ảnh đều đạt mức này. Ảnh vẽ tay trên web có thể khác phân phối MNIST nên hiển thị confidence của mô hình thay cho cam kết đúng.

3.4. Suy luận, triển khai và phạm vi mở rộng

Checkpoint PyTorch được xuất thành trọng số JSON. API thực hiện convolution, ReLU, max-pooling, flatten và dense bằng đúng trọng số đã huấn luyện, dùng Python standard library để tránh đóng gói PyTorch nặng vào serverless. Bộ vector đối chiếu kiểm tra logits của API với PyTorch. Giao diện hỗ trợ ảnh test thật, canvas vẽ số, kiểm tra ảnh trống, trạng thái đang xử lý và thông báo lỗi.

ASL được giữ như ví dụ mở rộng của bài toán thị giác: các cử chỉ M/N hoặc U/V gần nhau, nền ảnh và góc chụp có thể làm giảm khả năng tổng quát hóa. Trong lần thực hiện này chưa huấn luyện lại ASL, vì vậy các accuracy và biểu đồ ASL cũ không được dùng như kết quả mới. Tương tự, không khẳng định Keras và PyTorch tương đương từ một phép thử chỉ chạy PyTorch.

CHƯƠNG 4. MẠNG NƠ-RON HỒI QUY RNN

4.1. Lý thuyết RNN

4.1.1. Vấn đề với dữ liệu tuần tự

Dense layer không giữ trạng thái qua thời gian. CNN có thể xử lý tín hiệu 1D theo trường tiếp nhận hữu hạn; RNN bổ sung trạng thái truyền qua các bước để biểu diễn ngữ cảnh của chuỗi.

• Giá hôm nay phụ thuộc vào giá hôm qua

• Từ trong câu phụ thuộc vào từ trước đó

• Nhịp tim tương lai phụ thuộc vào lịch sử

→ Cần kiến trúc có bộ nhớ (memory) giữa các bước thời gian.

4.1.2. RNN cơ bản — Vanilla RNN

Tại mỗi bước thời gian t:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$
$$\hat{y}_t = W_y h_t + b_y \quad (4.1)$$

Trong đó:

• xi: vector đầu vào tại bước t

• hi: hidden state (bộ nhớ ngắn hạn)

• Wx, Wh và Wγ là ma trận trọng số; các tham số được dùng chung tại mọi bước thời gian.

Backpropagation Through Time (BPTT):

$$\frac{\partial \mathcal{L}}{\partial W_h} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial W_h} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial h_T} \cdot \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k} \cdot \frac{\partial h_t}{\partial W_h} \quad (4.2)$$

4.1.3. LSTM — Long Short-Term Memory

LSTM thêm cell state ct như "bộ nhớ dài hạn" và 3 gates kiểm soát luồng thông tin:

Forget Gate: Quyết định quên bao nhiêu cell state cũ

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f) \quad (4.3)$$

Input Gate: Quyết định thêm thông tin mới nào

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad \tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \tag{4.4}$$

Cập nhật Cell State:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{4.5}$$

Output Gate: Quyết định đầu ra

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(c_t) \tag{4.6}$$

4.1.4. GRU — Gated Recurrent Unit

GRU đơn giản hóa LSTM bằng cách gộp forget gate và input gate thành update gate:

$$\begin{aligned} z_t &= \sigma(W_z[h_{t-1}, x_t] + b_z) \quad \text{(Update gate)} \\ r_t &= \sigma(W_r[h_{t-1}, x_t] + b_r) \quad \text{(Reset gate)} \\ \tilde{h}_t &= \tanh(W[r_t \odot h_{t-1}, x_t] + b) \quad \text{(Candidate)} \\ h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \end{aligned} \tag{4.7}$$

Bảng 4.1. So sánh cấu trúc recurrent network

Mạng	Trạng thái	Số cổng	Đặc điểm
RNN	Hidden state ht	Không có cổng	Ít tham số, khó học phụ thuộc xa
LSTM	ht và cell state ct	Input, forget, output	Luồng cộng qua cell state hỗ trợ truyền gradient
GRU	Hidden state ht	Reset và update	Ít tham số hơn LSTM cùng kích thước ẩn

4.1.5. Cửa sổ trượt và các kiểu đầu vào đầu ra

Cửa sổ trượt tạo một mẫu bằng L quan sát quá khứ và nhãn của bước kế tiếp. Many-to-one ánh xạ chuỗi thành một giá trị hoặc nhãn; one-to-many sinh chuỗi từ một ngữ cảnh đầu; many-to-many tạo một đầu ra ở mỗi bước hoặc một chuỗi đầu ra khác độ dài. Thực nghiệm dự báo giá trong chương sử dụng many-to-one.

$$X_t = [r_{t-L+1}, \ldots, r_t], r_t = \log(P_t / P_{t-1}), \hat{P}_{t+1} = P_t e^{\hat{r}_{t+1}} \tag{4.8}$$

Pt là giá đóng cửa, rt là log-return, L=30 là chiều dài cửa sổ. Dự báo tại t dùng dữ liệu không muộn hơn t; nhãn ở t+1. Mỗi cửa sổ khởi tạo trạng thái ẩn bằng 0 và không truyền hidden state từ test về train.

4.1.6. BPTT và ổn định gradient

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{j=t-k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \quad (4.9)$$

BPTT trải mạng theo thời gian và cộng gradient của cùng tham số qua các bước. Tích nhiều Jacobian có chuẩn nhỏ hơn 1 có thể làm gradient suy giảm, còn chuẩn lớn có thể gây bùng nổ. LSTM/GRU tạo đường truyền có cổng để giảm khó khăn này nhưng không bảo đảm giải quyết mọi phụ thuộc dài hạn. Gradient clipping giới hạn norm; chuẩn hóa chỉ trên train và chọn checkpoint bằng validation giúp ổn định thực nghiệm.

4.2. Dữ liệu, phép chia và cấu hình

Dùng snapshot Yahoo Finance của Tiểu luận 02: FPT.VN có 1.304 phiên và vàng futures GC=F có 1.257 phiên, từ 15/10/2021 đến 14/10/2026. GC=F không được đổi tên thành spot XAU/USD. Lợi suất được tính theo phiên có thật, không nội suy ngày nghỉ. Dữ liệu e-commerce không được đưa vào thực nghiệm mới vì chưa xác minh được thứ tự sự kiện và nhãn đủ để biện minh cho RNN.

Ngày của nhãn quyết định tập dữ liệu: FPT train trước 13/02/2026, validation từ 13/02 đến trước 04/06/2026; vàng train trước 26/02/2026, validation đến trước 09/06/2026. Test kéo dài đến nhãn 14/10/2026. Cách định nghĩa này khác chỉ số ngày đặc trưng của bảng thị trường ở Chương 5; không so sánh trực tiếp hai bảng như cùng tập test. Validation/test được phép dùng lịch sử đã biết trước ngày nhãn, phù hợp dự báo cuốn chiếu một bước.

Bảng 4.2. Phân chia chuỗi theo ngày nhãn

Miền	Train / Val / Test	Khoảng ngày nhãn test
FPT	1099 / 79 / 95	2026-06-04 — 2026-10-14
Gold	1066 / 71 / 89	2026-06-09 — 2026-10-14

Ba mạng dùng 16 hidden units, một recurrent layer và một linear head, Adam learning_rate=0,001, 35 epoch, batch_size=128 và seed=42. Trung bình và độ lệch chuẩn của đầu vào/nhãn chỉ tính trên train. Checkpoint tối thiểu validation MSE được lưu; mạng triển khai được chọn bằng validation MAE theo đơn vị giá, có Persistence cùng tham gia. Test được giữ cho báo cáo cuối.

Ở phần cài đặt, GRU dùng quy ước PyTorch: reset gate nhân sau phép biến đổi tuyến tính của hidden state trong nhánh tạo trạng thái ứng viên. Đây là một biến thể so với

cách đặt reset gate trước ma trận trong một số trình bày lý thuyết. API tuân theo đúng quy ước của checkpoint; kiểm thử đối chiếu cả sáu mạng giúp phát hiện sai khác thứ tự cổng hoặc cách áp dụng reset gate.

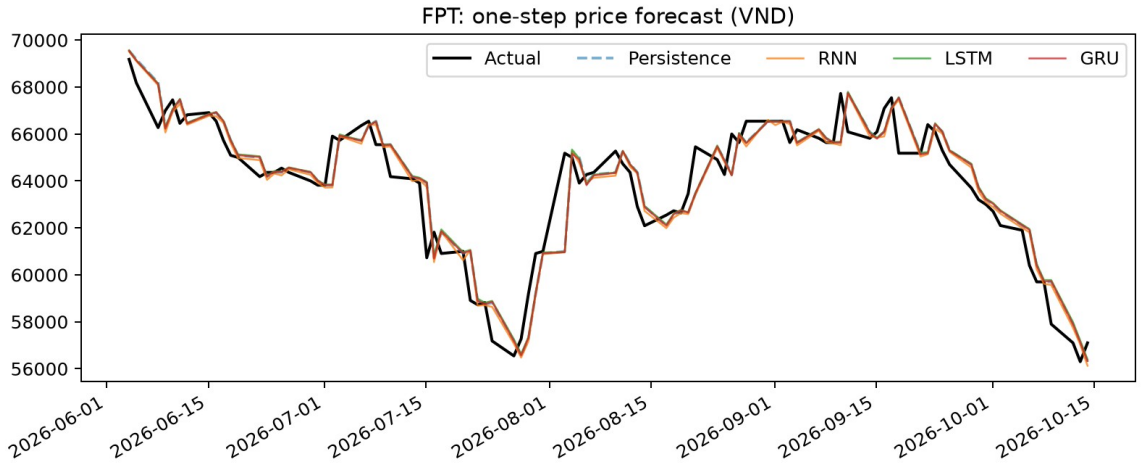
4.3. So sánh RNN, LSTM, GRU và Persistence

Bảng 4.3. Kết quả mạng chuỗi trong đơn vị gốc VND hoặc USD/oz

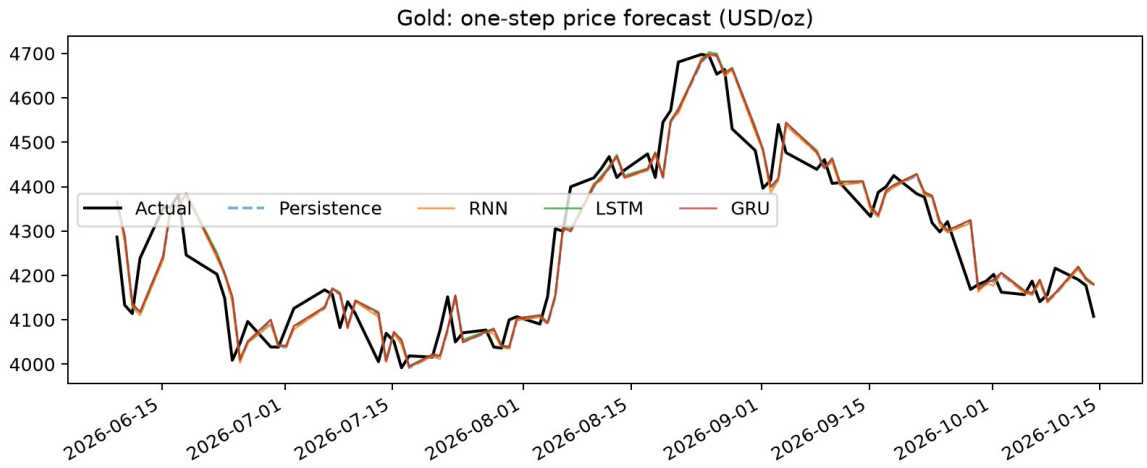
Miền / Mạng	Tham số	Val MAE	Test MAE	Test RMSE
FPT / RNN	321	1270.880	720.604	1032.599
FPT / LSTM	1233	1271.182	745.759	1050.320
FPT / GRU	929	1265.027	737.054	1044.655
Gold / RNN	321	63.676	47.342	61.988
Gold / LSTM	1233	63.707	48.168	63.044
Gold / GRU	929	63.622	48.097	62.834

FPT: Persistence test MAE 734.258 VND. Mạng có validation MAE thấp nhất là GRU (1265.027), trong khi Persistence validation MAE 1266.974. Vì vậy quy tắc trước test chọn GRU để trình diễn trên web.

GOLD: Persistence test MAE 48.089 USD/oz. Mạng có validation MAE thấp nhất là GRU (63.622), trong khi Persistence validation MAE 63.152. Vì vậy quy tắc trước test chọn Persistence để trình diễn trên web.



Hình 4.1. Dự báo một bước FPT bằng ba mạng và baseline



Hình 4.2. Dự báo một bước vàng futures GC=F

Vanilla RNN có MAE test thấp nhất trong ba mạng ở cả hai miền trong lần chạy này, nhưng không được đổi lựa chọn triển khai sau khi nhìn test. Kết quả bác bỏ việc mặc định LSTM luôn tốt hơn. Chuỗi ngắn, ít dữ liệu, nhiễu thị trường và một random seed hạn chế khả năng khái quát hóa. Mức chênh MAE nhỏ chưa chứng minh lợi ích giao dịch; báo cáo không suy ra lợi nhuận.

4.4. Hạn chế và khả năng mở rộng

Thực nghiệm là dự báo một bước trên snapshot cố định, không phải dự báo nhiều bước tương lai hoặc dữ liệu trực tiếp. Hướng mở rộng gồm nhiều seed, walk-forward backtest, khoảng tin cậy theo khối, nhiều chiều dài cửa sổ và covariate có timestamp. Cần khóa ngân sách tuning trước khi thử, tránh biến test thành validation qua nhiều lần sửa mô hình.

Mô hình triển khai giữ checkpoint train lịch sử, không refit bằng test để giữ được dấu vết đánh giá. Người dùng có thể sửa giá gần nhất và 30 log-return để thử kịch bản; kết quả không có khoảng tin cậy đã hiệu chỉnh cho kịch bản ngoài miền. Checkpoint của mọi RNN/LSTM/GRU và training history được lưu riêng để kiểm tra.

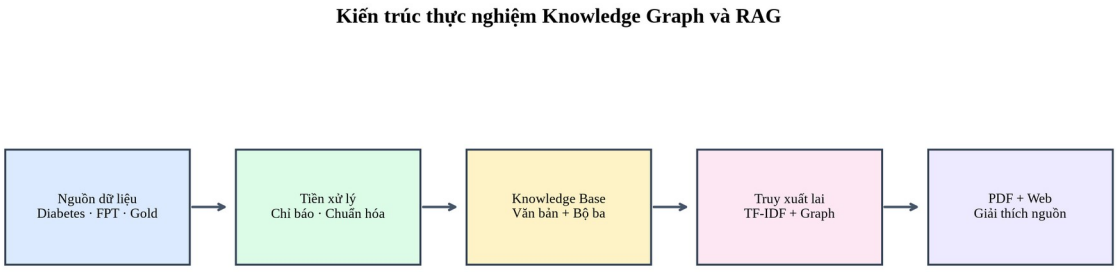
CHƯƠNG 5. KNOWLEDGE GRAPH VÀ RAG

5.1. MỞ ĐẦU

Knowledge Graph (KG) và Retrieval-Augmented Generation (RAG) xuất phát từ hai truyền thống khác nhau. KG nhấn mạnh biểu diễn tường minh: thực thể nào tồn tại, chúng liên hệ ra sao và ràng buộc nào phải được giữ. RAG xuất phát từ nhu cầu cho mô hình sinh tiếp cận một kho tri thức ngoài tại thời điểm trả lời. Khi ghép hai hướng này, hệ thống vừa có khả năng tìm kiếm mềm theo ngôn ngữ, vừa có cấu trúc để kiểm tra đường suy luận.

Bài làm lựa chọn kiến trúc có thể chạy trọn trong một notebook. TF-IDF đảm nhiệm truy xuất văn bản, NetworkX lưu đồ thị, còn bộ tổng hợp ghép bằng chứng tìm được thành câu trả lời. Dữ liệu FPT.VN và GC=F được đọc từ snapshot đã khóa trong lần tái lập; có chế độ tải mới khi được bật; dữ liệu tiểu đường lấy từ scikit-learn.

Ba miền được chọn tạo ra phép thử đủ khác biệt. Y tế yêu cầu diễn đạt thận trọng; chứng khoán là chuỗi thời gian nội địa; vàng chịu tác động của yếu tố vĩ mô và được niêm yết quốc tế. Một pipeline dùng chung phải giữ được ngữ cảnh riêng của từng miền mà không trộn lẫn bằng chứng.



Hình 5.1. Kiến trúc KG và RAG

5.1.1. LỊCH SỬ PHÁT TRIỂN CỦA KNOWLEDGE GRAPH

Ý tưởng nối các khái niệm thành mạng ngữ nghĩa đã xuất hiện từ những nghiên cứu ban đầu về trí nhớ và biểu diễn tri thức. Sau đó, hệ chuyên gia đưa luật IF-THEN vào các miền hẹp. Bước ngoặt của Web ngữ nghĩa là việc chuẩn hóa RDF, RDFS, OWL và SPARQL, khiến bộ ba chủ thể-vị từ-đối tượng trở thành một ngôn ngữ trao đổi máy đọc được,.

Năm 2012, cách gọi Knowledge Graph được phổ biến rộng rãi qua công cụ tìm kiếm của Google. Trong doanh nghiệp, KG tiếp tục được dùng cho danh mục sản phẩm,

phát hiện gian lận, quản trị dữ liệu chủ và gợi ý nội dung. Điểm chung không phải một loại cơ sở dữ liệu cụ thể, mà là cách coi quan hệ giữa các thực thể là công dân hạng nhất.

Ở bài này, đồ thị được cài bằng Labeled Property Graph đơn giản. Mỗi nút có type và domain; mỗi cạnh có nhãn quan hệ. Lựa chọn này gần với cách Neo4j tổ chức dữ liệu và thuận tiện cho trực quan hóa, dù chưa khai thác đầy đủ suy luận ontology như OWL.

5.1.2. LỊCH SỬ PHÁT TRIỂN CỦA RAG

Trước RAG, các hệ hỏi đáp mở thường tách bộ tìm kiếm tài liệu và bộ đọc hiểu. REALM đưa cơ chế truy xuất vào quá trình tiền huấn luyện. Lewis và cộng sự sau đó định danh kiến trúc Retrieval-Augmented Generation, kết hợp bộ truy xuất dense với mô hình sinh seq2seq cho nhiệm vụ giàu tri thức.

Sự phổ biến của mô hình ngôn ngữ lớn khiến RAG trở thành một mẫu triển khai thực dụng: tài liệu có thể cập nhật mà không cần huấn luyện lại toàn bộ mô hình, đồng thời câu trả lời giữ được liên kết về nguồn. Tuy nhiên, vector search thường ưu tiên đoạn giống câu hỏi và có thể bỏ qua quan hệ gián tiếp. GraphRAG giải quyết phần thiếu này bằng việc xây thực thể, quan hệ, cụm và tóm tắt theo cấu trúc.

Nguyên mẫu trong notebook không cố tái tạo toàn bộ Microsoft GraphRAG. Nó giữ hai ý cốt lõi: truy xuất đồng thời từ văn bản và đồ thị, sau đó đưa cả hai nhóm bằng chứng vào bước tổng hợp.

5.1.3. VAI TRÒ VÀ Ý NGHĨA TRONG AI

KG giúp mô hình có từ điển thực thể thống nhất, quan hệ có tên và đường đi có thể kiểm tra. RAG giúp tri thức ngoài đi vào ngữ cảnh trả lời đúng thời điểm. Kết hợp hai thành phần làm giảm nguy cơ câu trả lời chỉ dựa trên tương đồng bề mặt, nhưng không đồng nghĩa loại bỏ hoàn toàn sai sót.

Trong y tế, giá trị quan trọng nhất là provenance: người đọc phải biết nhận xét dựa trên biến nào và nguồn nào. Trong tài chính, dữ liệu có tính thời điểm nên ngày quan sát phải đi cùng con số. Trong cả hai trường hợp, câu trả lời nên phân biệt sự kiện đo được, quan hệ thường gặp và kết luận mang tính dự báo.

Ý nghĩa học thuật của bài nằm ở khả năng chạy lại. Người chấm có thể Run All, xem dữ liệu thô đã lưu, đối chiếu bộ ba, kiểm tra tài liệu được truy xuất và mở PDF được sinh từ cùng một trạng thái chạy.

5.1.4. PHẠM VI, CÂU HỎI VÀ ĐÓNG GÓP

Bài đặt ba câu hỏi: (1) một cấu trúc KG thống nhất có mô tả được ba miền khác nhau không; (2) truy xuất lai có cung cấp nhiều bằng chứng hơn vector search đơn lẻ không; và (3) có thể đóng gói toàn bộ quy trình trong một notebook duy nhất không.

Đóng góp thực hành gồm lớp KnowledgeGraphCore, bộ tải Yahoo Finance không cần API key, các bước tạo chỉ báo, VectorRetriever, truy xuất đồ thị theo miền, đánh giá precision@3, ứng dụng dự đoán, chatbot sức khỏe có nguồn và bộ sinh PDF. Mọi tệp phụ đều được tạo lại từ notebook.

Ba nguồn dữ liệu được mô tả bằng số dòng, số cột, khoảng thời gian, đơn vị và biến đích. Bảng được sinh trực tiếp từ DataFrame sau xử lý nên khớp với lần chạy hiện tại.

Mô tả ba nguồn dữ liệu tại thời điểm chạy

Miền	Nguồn	Số dòng	Số cột	Phạm vi	Biến đích/đơn vị
Tiểu đường	scikit-learn load_diabetes	442	11	442 mẫu nghiên cứu	tiến triển sau 1 năm
Chứng khoán	Yahoo Finance chart API - FPT.VN	1304	10	15/10/2021 - 14/10/2026	Close phiên kế tiếp
Giá vàng	Yahoo Finance chart API - GC=F	1257	10	15/10/2021 - 14/10/2026	Close phiên kế tiếp

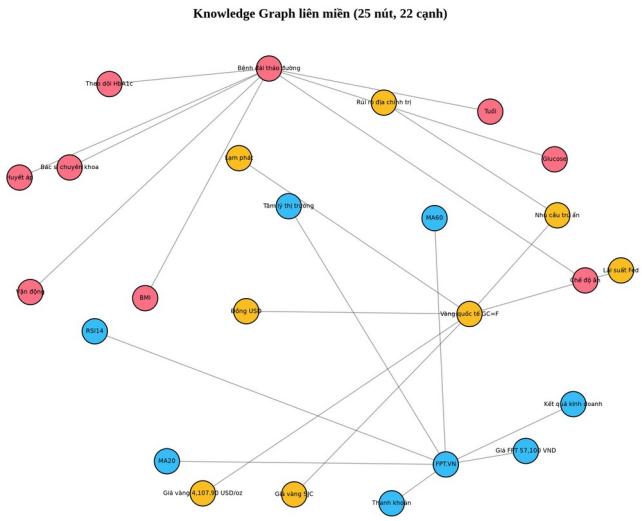
Hình 5.2. Thông tin ba miền dữ liệu

5.2. KNOWLEDGE GRAPH - KNOWLEDGE BASE

Knowledge Base trong bài gồm hai lớp. Lớp văn bản giữ các đoạn mô tả dữ liệu, phương pháp và kết quả theo từng miền. Lớp đồ thị giữ thực thể cùng các quan hệ có hướng. Hai lớp dùng chung nhãn domain để ngăn truy xuất nhầm giữa tiểu đường, cổ phiếu và vàng.

Đồ thị tại lần chạy này có 25 nút và 22 cạnh. Kích thước nhỏ giúp người đọc kiểm tra thủ công. Khi mở rộng, cùng schema có thể chuyển sang Neo4j hoặc một triple store mà không thay đổi ý nghĩa bộ ba.

Knowledge Base không chỉ chứa kiến thức tĩnh. Các nút quan sát như giá FPT hoặc giá vàng gần nhất được tạo từ dữ liệu tải về. Cách làm này nối câu trả lời với đúng trạng thái thị trường tại thời điểm chạy.



Hình 5.3. Đồ thị tri thức đa miền

5.2.1. BỘ BA, THỰC THỂ VÀ QUAN HỆ

Một bộ ba có dạng (s, p, o), trong đó s là thực thể nguồn, p là quan hệ có tên và o là thực thể hoặc giá trị đích. Ví dụ: (MA20, MÔ_TẢ_XU_HƯỚNG_NGẮN_HẠN_CỦA, FPT.VN). Bộ ba này không khẳng định xu hướng sẽ tiếp diễn; nó chỉ diễn tả vai trò của chỉ báo.

Thực thể được gắn type như Condition, Biomarker, Stock, TechnicalIndicator, Commodity hoặc MacroFactor. Thuộc tính domain tách ba không gian nghiệp vụ. Tên quan hệ dùng động từ viết hoa để người đọc có thể đọc thành câu.

Việc chọn quan hệ là công đoạn có tính chủ quan. Báo cáo ưu tiên từ CÓ_THỂ_TÁC_ĐỘNG hoặc CÓ_QUAN_HỆ... thay vì GÂY_RA khi dữ liệu không đủ để chứng minh nhân quả.

5.2.2. KNOWLEDGE GRAPH CORE ENGINE

Lớp KnowledgeGraphCore bọc MultiDiGraph của NetworkX. add_entity quản lý nút và thuộc tính; add_relation thêm cạnh có hướng; neighbors trả về bằng chứng một bước; multi_hop tìm các đường đơn trong giới hạn số hop. Cấu trúc MultiDiGraph cho phép hai thực thể có nhiều loại quan hệ.

Độ phức tạp của truy vấn lân cận tỷ lệ với bậc của nút. Truy vấn mọi đường đơn có thể tăng nhanh theo kích thước đồ thị, vì vậy max_hops phải được giới hạn. Với đồ thị sản xuất, cần chỉ mục, phân quyền và chiến lược truy vấn cụ thể thay vì duyệt toàn bộ.

Toàn bộ cạnh được xuất thành knowledge_graph_triples.csv. Đây là điểm kiểm tra thuận tiện: mỗi dòng trong tệp phải đọc được như một phát biểu, không có nút mồ côi vô nghĩa và không chứa con số không rõ ngày.

KnowledgeGraphCore: thực thể, quan hệ và multi-hop Tieuluan02_06_Nhom2_Giangkt.ipynb

```
class KnowledgeGraphCore:
    def __init__(self):
        self.graph = nx.MultiDiGraph()

    def add_entity(self, entity_id, entity_type="Entity", **properties):
        self.graph.add_node(entity_id, type=entity_type, **properties)

    def add_relation(self, subject, predicate, obj, weight=1.0, **properties):
        if subject not in self.graph: self.add_entity(subject)
        if obj not in self.graph: self.add_entity(obj)
        self.graph.add_edge(subject, obj, key=predicate,
                            relation=predicate, weight=float(weight), **properties)

    def multi_hop(self, start, max_hops=2):
        paths = []
        for target in self.graph.nodes:
            if target != start:
                paths.extend(nx.all_simple_paths(self.graph, start, target,
    cutoff=max_hops))
        return paths
```

Hình 5.4. Cấu trúc KnowledgeGraphCore

5.2.3. RDF VÀ LABELED PROPERTY GRAPH

RDF phù hợp khi cần URI toàn cục, liên kết dữ liệu và suy luận theo chuẩn W3C. Labeled Property Graph thuận tiện khi cạnh cần thuộc tính và truy vấn đường đi là trọng tâm. Bài chọn mô hình thứ hai vì mục tiêu là nguyên mẫu phân tích và trực quan hóa bằng Python.

Nếu chuyển sang RDF, các entity_id cần ánh xạ thành IRI; relation thành predicate; giá trị số thành literal có datatype. Nếu chuyển sang Neo4j, type có thể thành node label, relation thành relationship type và các trường domain/source thành property.

Không có mô hình nào tốt tuyệt đối. Tiêu chí chọn là khả năng tích hợp với hệ thống, yêu cầu suy luận, quy mô và kỹ năng vận hành của nhóm.

5.2.4. CHẤT LƯỢNG VÀ QUẢN TRỊ TRI THỨC

Một KG hữu ích cần hơn số lượng cạnh. Bốn kiểm tra tối thiểu gồm: tính duy nhất của định danh, tính hợp lệ của miền-phạm vi, nguồn gốc quan hệ và thời hạn của dữ liệu động. Các quan hệ y tế và tài chính phải có mức tin cậy hoặc câu chữ thể hiện độ bất định.

Bài hiện lưu source ở lớp tài liệu và domain ở lớp đồ thị. Bản sản xuất nên bổ sung created_at, valid_from, valid_to, confidence và evidence_id cho từng cạnh. Khi đó, một quan hệ đã cũ có thể bị loại trước khi đưa vào prompt.

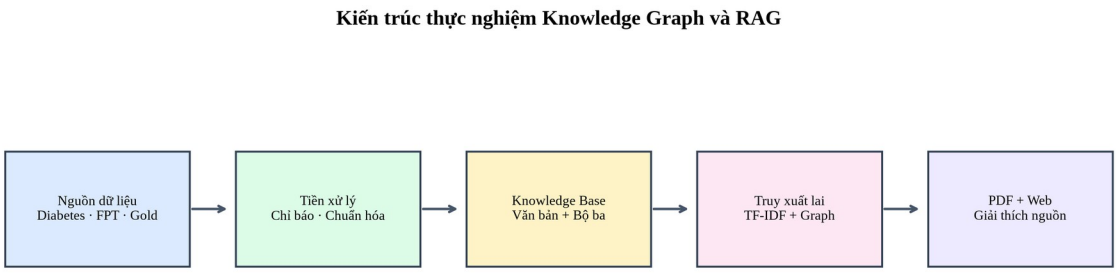
Kiểm soát chất lượng cần vòng phản hồi của chuyên gia. Bác sĩ xác nhận ontology y tế; chuyên viên tài chính rà soát khái niệm và đơn vị; kỹ sư dữ liệu kiểm tra schema cùng pipeline cập nhật.

5.3. RAG - RETRIEVAL-AUGMENTED GENERATION

Pipeline RAG gồm ingestion, indexing, retrieval và generation. Văn bản được chia thành các đơn vị có nguồn; TF-IDF biến chúng thành vector; cosine similarity xếp hạng theo câu hỏi. Bước sinh ghép hai đoạn phù hợp nhất với các bộ ba liên quan.

TF-IDF được chọn vì minh bạch, nhẹ và tái lập. Hạn chế là không hiểu đồng nghĩa sâu như embedding. Để giảm nhược điểm, câu hỏi được nhận diện miền bằng một từ điển alias và GraphRAG bổ sung các quan hệ từ đồ thị.

Mỗi câu trả lời trả về cả documents, score, triples và domain. Cấu trúc này quan trọng hơn phần câu chữ: giao diện hoặc báo cáo có thể trình bày bằng chứng mà không phải phân tích ngược chuỗi văn bản.



Hình 5.5. Các bước truy xuất và tổng hợp bằng chứng

5.3.1. NẠP, LẬP CHỈ MỤC VÀ TRUY XUẤT

Các tài liệu ngắn được xây từ mô tả bộ dữ liệu và kết quả thực nghiệm. Trong hệ thống lớn, ingestion cần đọc PDF, HTML hoặc cơ sở dữ liệu; chuẩn hóa Unicode; bỏ trùng; chia đoạn theo ngữ nghĩa; và gắn metadata trước khi embedding.

TfidfVectorizer dùng unigram và bigram. Bigram giúp phân biệt các cụm như giá vàng, tiểu đường, lãi suất Fed. top_k=3 giữ ngữ cảnh ngắn và dễ kiểm tra. Precision@3 theo domain được dùng như phép đo đơn giản cho độ đúng của truy xuất.

Điểm cosine chỉ phản ánh độ gần từ vựng, không phải độ đúng. Vì vậy báo cáo không diễn giải score như xác suất và không tự động loại bỏ nguồn chuyên môn chỉ vì score thấp.

VectorRetriever dùng TF-IDF và cosine similarity Tieuluan02_06_Nhom2_Giangkt.ipynb

```
class VectorRetriever:
    def __init__(self, docs):
        self.docs = docs
        self.vectorizer = TfidfVectorizer(
            lowercase=True, ngram_range=(1, 2), min_df=1,
            token_pattern=r"(?u)\b\w+\b"
        )
        self.matrix = self.vectorizer.fit_transform([d["text"] for d in docs])

    def search(self, query, top_k=3):
        query_vector = self.vectorizer.transform([query])
        scores = cosine_similarity(query_vector, self.matrix).ravel()
        order = np.argsort(scores)[::-1][:top_k]
        return [{**self.docs[i], "score": float(scores[i])} for i in order]
```

Hình 5.6. Lập chỉ mục và truy xuất TF-IDF

5.3.2. TRUY XUẤT ĐỒ THỊ VÀ SUY LUẬN NHIỀU BƯỚC

graph_evidence nhận diện miền, chọn các nút cùng domain hoặc trùng từ khóa, rồi lấy quan hệ vào và ra. Phương án này đủ cho đồ thị nhỏ. Khi đồ thị lớn, cần entity linking, tìm kiếm theo embedding của nhãn và truy vấn Cypher có giới hạn.

Suy luận nhiều bước được minh họa bởi chuỗi Rủi ro địa chính trị → Nhu cầu trú ẩn → Vàng quốc tế. Vector search có thể tìm một đoạn nói về địa chính trị nhưng không mặc nhiên biểu diễn hai quan hệ riêng. Đồ thị giúp giữ cấu trúc của lời giải thích.

Đường đi chỉ là bằng chứng cấu trúc, không tự chứng minh nhân quả. Mọi kết luận vẫn phụ thuộc chất lượng cạnh và nguồn đã dùng để tạo cạnh.

GraphRAG: ghép bằng chứng văn bản và bộ ba Tieuluan02_06_Nhom2_Giangkt.ipynb

```
def graphrag_answer(query, top_k=3):
    docs = retriever.search(query, top_k=top_k)
    triples = graph_evidence(query)
    domain = detect_domain(query)
    domain_docs = [d for d in docs if d["domain"] in (domain, "method")]
    statements = [d["text"] for d in domain_docs[:2]]
    graph_text = [f"{s} -[{p}]-> {o}" for s, p, o in triples[:3]]
    answer = " ".join(statements)
    if graph_text:
        answer += " Quan hệ đồ thị: " + "; ".join(graph_text)
    return {"answer": answer, "documents": domain_docs, "triples": triples}
```

Hình 5.7. Truy xuất bằng chứng đồ thị

5.3.3. TỔNG HỢP CÂU TRẢ LỜI CÓ CĂN CỨ

Bộ sinh trong bài là deterministic synthesizer. Nó chọn hai tài liệu phù hợp, nối tối đa ba quan hệ đồ thị và thêm cảnh báo theo miền. Kết quả ít tự nhiên hơn LLM nhưng có lợi thế là chạy offline và không tự thêm sự kiện ngoài bằng chứng.

Có thể thay bộ sinh bằng một LLM cục bộ hoặc dịch vụ API. Khi đó prompt phải tách rõ QUESTION, TEXT EVIDENCE, GRAPH EVIDENCE và yêu cầu từ chối nếu bằng chứng thiếu. Đầu ra nên ở dạng JSON để kiểm tra citation_id.

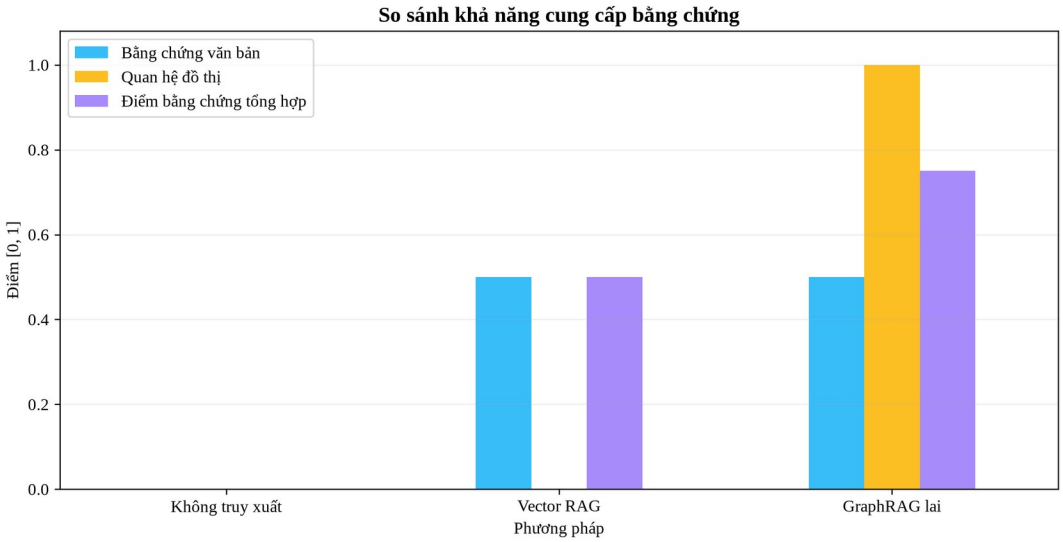
Thiết kế production cần chống prompt injection từ tài liệu, lọc dữ liệu nhạy cảm, theo dõi phiên bản chỉ mục và ghi log các nguồn đã đưa cho mô hình.

5.3.4. ĐÁNH GIÁ RAG VÀ GRAPHRAG

Bộ kiểm thử gồm sáu câu hỏi, hai câu cho mỗi miền. vector_precision@3 đo tỷ lệ tài liệu thuộc đúng miền trong ba kết quả đầu. graph_evidence ghi nhận hệ thống có tìm được cạnh liên quan hay không. Điểm tổng hợp cộng một phần thưởng nhỏ khi cả hai kênh đều có bằng chứng.

Ở lần chạy này, precision@3 trung bình là 0.500; tỷ lệ câu hỏi có bằng chứng đồ thị là 1.000; điểm bằng chứng lai là 0.750.

Đây là smoke test, chưa phải benchmark khoa học. Một đánh giá đầy đủ cần nhiều câu hỏi do người độc lập soạn, đáp án chuẩn, chấm faithfulness, answer relevance, citation correctness, độ trễ và chi phí.



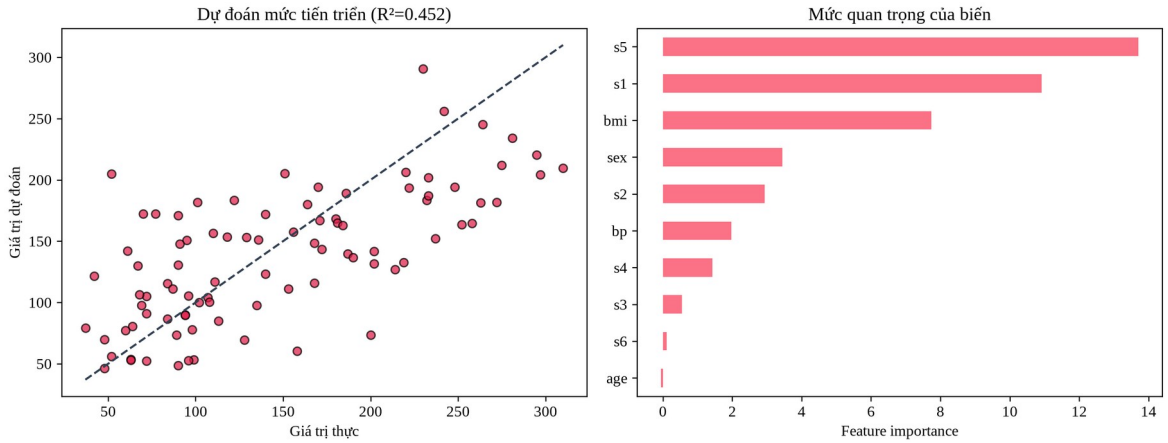
Hình 5.8. Đánh giá smoke test RAG và GraphRAG

5.4. KB VÀ RAG CHO BỆNH TIỂU ĐƯỜNG

Dữ liệu diabetes gồm 442 người đã mắc bệnh, 10 thuộc tính nền và điểm tiến triển sau một năm. Đầu ra là hồi quy nghiên cứu, không phải xác suất mắc tiểu đường.

Dữ liệu được chia thành 282 mẫu train, 71 mẫu hiệu chỉnh khoảng dự báo và 89 mẫu test. K-fold 5 phần chỉ chạy trên train; so sánh Median baseline, Linear, Ridge, ElasticNet, Huber, SVR RBF, Random Forest và Gradient Boosting. Thêm mô hình phi tuyến không đồng nghĩa chắc chắn tốt hơn; quyết định vẫn dựa trên CV MAE.

Mô hình được chọn theo CV: Ridge; CV MAE 44.44; test MAE 42.94; R2 0.452. MAE của dự báo trung vị là 62.28. Khoảng dự báo mục tiêu 90% có độ bao phủ test 86.5%.



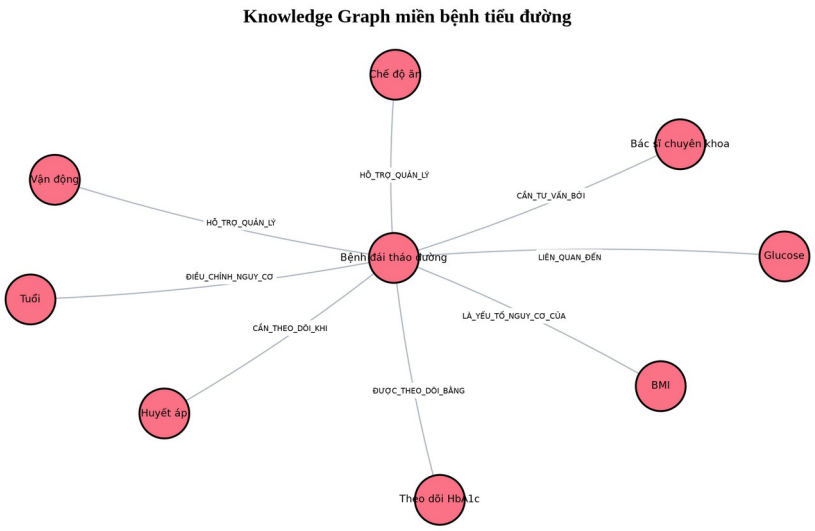
Hình 5.9. Mô hình hồi quy tiến triển tiểu đường

5.4.1. ONTOLOGY VÀ XỬ LÝ DỮ LIỆU Y TẾ

Ontology tối giản gồm Condition, Biomarker, RiskFactor, Lifestyle, Monitoring và Professional. Các quan hệ như LIÊN_QUAN_ĐẾN hoặc HỖ_TRỢ_QUẢN_LÝ được chọn để tránh biến một tương quan thành chỉ định điều trị.

Dữ liệu không có tên bệnh nhân hay định danh cá nhân. Trong tình huống thực, hồ sơ y tế cần mã hóa, phân quyền theo vai trò, nhật ký truy cập và quy trình chấp thuận. Không nên đưa thông tin nhận dạng vào dịch vụ embedding công cộng khi chưa có cơ sở pháp lý.

Mô hình Ridge được chọn theo MAE trung bình của 5 fold. Permutation importance được tính trên tập test để cùng một cách giải thích áp dụng được cho cả mô hình tuyến tính và mô hình cây.



Hình 5.10. Đồ thị tri thức y tế

5.4.2. THỰC NGHIỆM HỎI ĐÁP Y TẾ

Câu hỏi mẫu hỏi các yếu tố liên quan đến quản lý bệnh tiểu đường. Vector retriever tìm mô tả theo dõi đường huyết, HbA1c, huyết áp, cân nặng, ăn uống và vận động. Đồ thị bổ sung các cạnh nối bệnh với theo dõi và chuyên gia.

Câu trả lời sinh ra trong lần chạy: Theo dõi đường huyết, HbA1c, huyết áp, cân nặng, chế độ ăn và vận động là các thành phần thường gặp trong quản lý bệnh đái tháo đường; quyết định điều trị cần bác sĩ. Bộ dữ liệu diabetes của scikit-learn có 442 quan sát, 10 biến nền ở thang đo gốc và biến đích phản ánh mức tiến triển bệnh sau một năm. Quan hệ đồ thị liên quan: Bệnh đái tháo đường -[ĐƯỢC_THEO_DÕI_BẰNG]→ Theo dõi HbA1c; Bệnh đái tháo đường -[CẦN_TƯ_VẤN_BỞI]→ Bác sĩ chuyên khoa; Glucose -[LIÊN_QUAN_ĐẾN]→ Bệnh đái tháo đường. Kết quả chỉ có giá trị minh họa; cần tham khảo nhân viên y tế cho quyết định cá nhân.

Biểu đồ đối chiếu giá trị thực - dự đoán và feature importance cho thấy mức độ phù hợp cũng như biến nào được mô hình sử dụng nhiều nhất.

CV + calibration riêng cho mô hình sức khỏe Tieuluan02_06_Nhom2_Giangkt.ipynb

```
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=.2, random_state=42)
X_train, X_cal, y_train, y_cal = train_test_split(
    X_dev, y_dev, test_size=.2, random_state=42)
name, search, rows = select_model(
    diabetes_candidates, X_train, y_train,
    KFold(5, shuffle=True, random_state=42), "Tiểu đường")
model = search.best_estimator_
interval = calibrate(model, X_cal, y_cal, X_test, y_test)
pred = model.predict(X_test)
```

Hình 5.11. Huấn luyện và đánh giá mô hình y tế

5.4.3. GIỚI HẠN VÀ HƯỚNG MỞ RỘNG Y TẾ

Bộ dữ liệu nhỏ, không đại diện cho dân số Việt Nam và không chứa bối cảnh điều trị. Chỉ số đích mang ý nghĩa nghiên cứu, khác với chẩn đoán đái tháo đường trong thực hành. Vì vậy không thể suy rộng độ chính xác mô hình thành hiệu quả lâm sàng.

Hướng mở rộng là dùng dữ liệu đã khử định danh, xác định rõ biến kết cục, đánh giá theo nhóm tuổi/giới và mời chuyên gia lâm sàng kiểm tra ontology. Ngoài độ chính xác, cần đo calibration, fairness và khả năng giải thích.

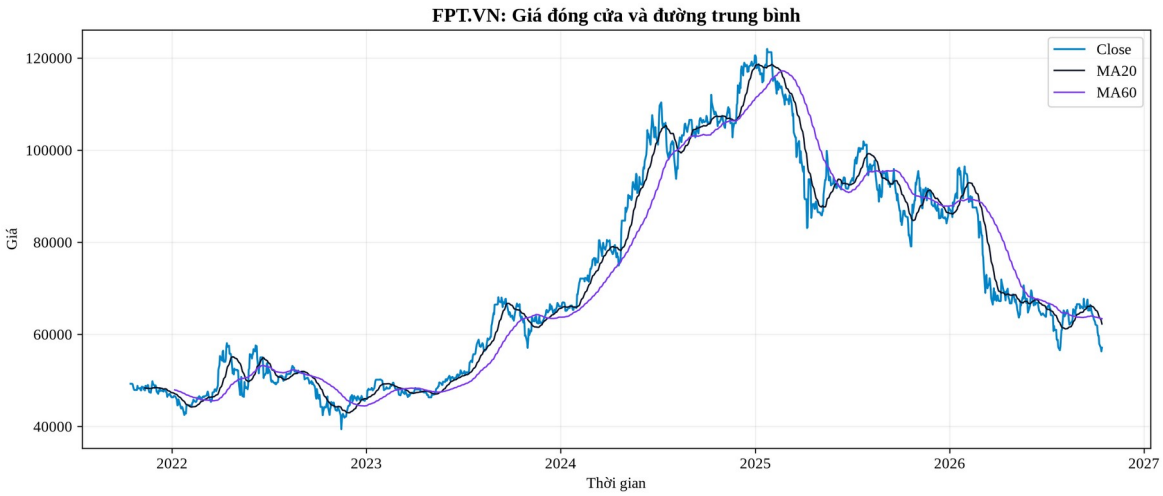
Với RAG y tế, mọi đoạn văn cần version và ngày hiệu lực. Hướng dẫn cũ phải được thu hồi khỏi chỉ mục thay vì chỉ thêm tài liệu mới.

5.5. KB VÀ RAG CHO CHỨNG KHOÁN

Dữ liệu FPT.VN được lấy từ Yahoo Finance chart API - FPT.VN. Khoảng thời gian 15/10/2021 - 14/10/2026 có 1304 quan sát. Giá đóng cửa gần nhất là 57,100 VND.

Lợi suất toàn kỳ là 15.91%, biến động năm hóa 27.63%, RSI14 8.2. Mỗi con số được gắn ngày chạy và nguồn dữ liệu.

Knowledge Graph nối FPT.VN với giá gần nhất, MA20, MA60, RSI, thanh khoản, kết quả kinh doanh và tâm lý thị trường. Các quan hệ mô tả vai trò chỉ báo, không tự động tạo khuyến nghị mua bán.



Hình 5.12. Chuỗi giá và chỉ báo FPT

5.5.1. DỮ LIỆU VÀ TIỀN XỬ LÝ FPT

Chuỗi OHLCV được sắp xếp tăng dần theo ngày; dòng thiếu Close bị loại. Return_1D, MA20, MA60, volatility20 và RSI14 được tính từ dữ liệu lịch sử. Đặc trưng bổ sung gồm thay đổi khối lượng, biên độ High-Low, chênh lệch Open-Close và các độ trễ một phiên.

Tại điểm cuối, MA20 = 62,260, MA60 = 63,396. Bộ học có 988 mẫu train và 85 mẫu test theo thứ tự thời gian.

Biến đích là giá Close của phiên kế tiếp. Việc dịch target một bước rồi drop dòng cuối giúp đặc trưng tại ngày t không chứa giá của ngày t+1.

Tải và tiền xử lý dữ liệu thị trường Tieuluan02_06_Nhom2_Giangkt.ipynb

```
def yahoo_chart(symbol, range_="5y", interval="1d"):
    url = f"https://query1.finance.yahoo.com/v8/finance/chart/{symbol}"
    response = requests.get(url, params={"range": range_, "interval": interval},
                            timeout=25, headers={"User-Agent": "Mozilla/5.0"})
    response.raise_for_status()
    result = response.json()["chart"]["result"][0]
    quote = result["indicators"]["quote"][0]
    frame = pd.DataFrame(quote, index=pd.to_datetime(result["timestamp"], unit="s"))
    frame.columns = [c.title() for c in frame.columns]
    return frame[["Open", "High", "Low", "Close", "Volume"]].dropna(subset=["Close"])

def add_indicators(df):
    out = df.copy().sort_index()
    out["Return_1D"] = out["Close"].pct_change()
    out["MA20"] = out["Close"].rolling(20).mean()
    out["MA60"] = out["Close"].rolling(60).mean()
    out["Volatility20"] = out["Return_1D"].rolling(20).std() * np.sqrt(252)
    return out
```

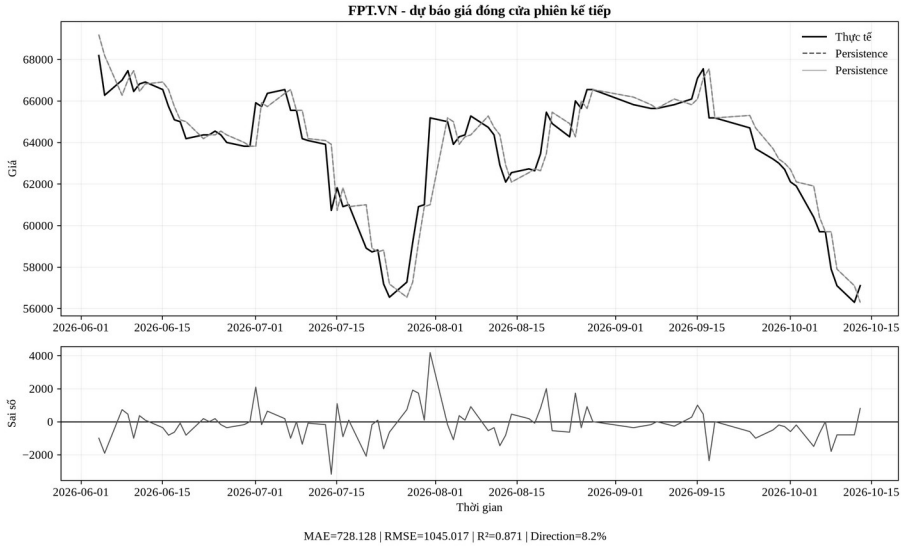
Hình 5.13. Pipeline xử lý dữ liệu thị trường

5.5.2. HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH FPT

Lịch sử tải được mở rộng từ 2 năm lên 5 năm. Chín họ cấu hình gồm Persistence, Ridge giá, Median lợi suất, Ridge lợi suất, Huber, SVR RBF, Random Forest và hai dạng Gradient Boosting. Tuning độ co lợi suất, cửa sổ học gần nhất và siêu tham số trên TimeSeriesSplit 5 phần, mỗi phần xác nhận 63 phiên, gap=1. Tập test giữ nguyên mốc của bản 2.0.

Chia theo thời gian: 988 train, 67 hiệu chỉnh, 85 test; bỏ một quan sát tại mỗi ranh giới. Mô hình Persistence đạt MAE 728.13, R2 0.871; baseline MAE 728.13. Cải thiện trên test 0.00%.

Test lịch sử từ 2026-06-04 đến 2026-10-13; MAE bản trước 728.13 VND. Bao phủ khoảng dự báo lịch sử: 97.6%. Kết quả hiện tại chưa cho thấy mô hình học máy thắng Persistence. Dữ liệu chuỗi thời gian không thỏa giả định trao đổi như mẫu độc lập nên mức 90% chỉ là mục tiêu, không phải bảo đảm cho tương lai.



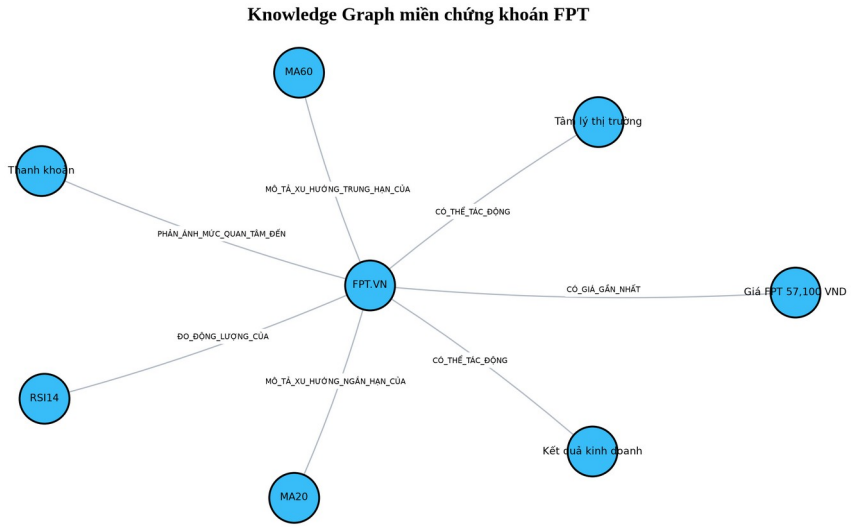
Hình 5.14. Dự báo FPT so với Persistence

5.5.3. HỎI ĐÁP CHỨNG KHOÁN VỚI GRAPHRAG

Câu trả lời mẫu: FPT.VN có MA20 62,260, MA60 63,396, RSI14 8.2 và biến động năm hóa 27.6%. Đường trung bình động mô tả xu hướng quá khứ chứ không bảo đảm giá tương lai. RSI là chỉ báo động lượng và cần kết hợp thanh khoản, cơ bản doanh nghiệp cùng bối cảnh thị trường. Quan hệ đồ thị liên quan: FPT.VN -[CÓ_GIÁ_GẦN_NHẤT]→ Giá FPT 57,100 VND; MA20 -[MÔ_TẢ_XU_HƯỚNG_NGẮN_HẠN_CỦA]→ FPT.VN; MA60 -[MÔ_TẢ_XU_HƯỚNG_TRUNG_HẠN_CỦA]→ FPT.VN. Đây là phân tích dữ liệu quá khứ, không phải khuyến nghị mua hoặc bán.

Tài liệu cung cấp số liệu động; đồ thị giải thích vai trò của chỉ báo và liên kết multi-hop. Bộ lọc domain ngăn tài liệu vàng hoặc y tế đi vào câu trả lời chứng khoán.

Kết quả mô hình được ghi thành tài liệu trong KB với model name, thời gian, MAE và R2. Vì vậy RAG có thể dẫn lại đúng lần huấn luyện thay vì tạo ra độ chính xác không tồn tại.



Hình 5.15. Đồ thị tri thức chứng khoán

5.5.4. RỦI RO VÀ HƯỚNG MỞ RỘNG

Corporate actions, thanh khoản, cách điều chỉnh giá và survivorship bias có thể làm sai phân tích. Yahoo Finance phù hợp bài tập nhưng nên đối chiếu nguồn chính thức nếu dùng trong nghiên cứu giao dịch.

Dự báo Close phiên kế tiếp thường có R2 cao do giá hôm nay gần giá ngày mai; vì vậy phải đọc cùng persistence baseline và directional accuracy. R2 cao không đồng nghĩa chiến lược có lợi nhuận.

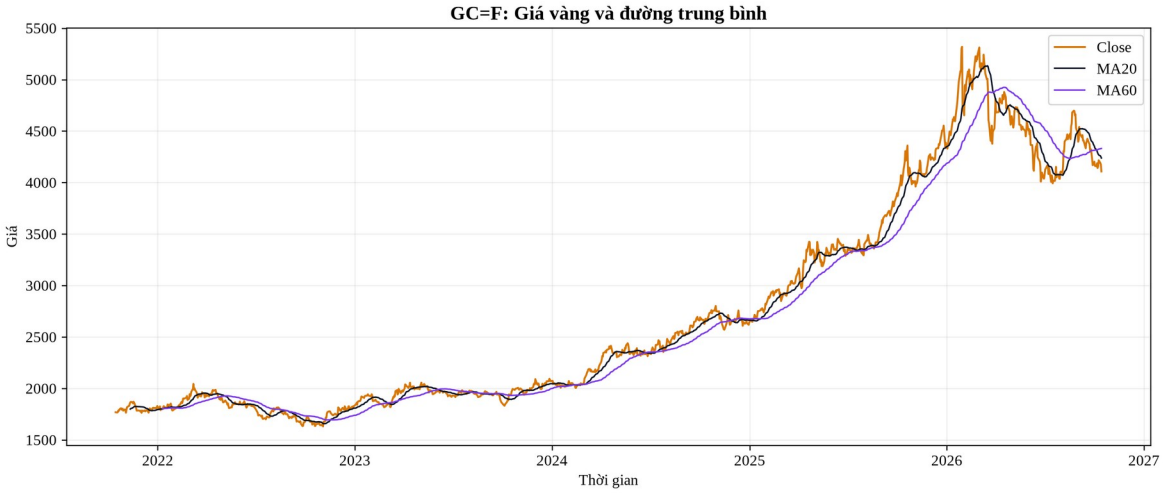
Hướng mở rộng là walk-forward backtest có phí, trượt giá, khoảng tin cậy và kiểm tra thay đổi chế độ thị trường. Mọi đầu ra vẫn cần cảnh báo không phải khuyến nghị đầu tư.

5.6. KB VÀ RAG CHO GIÁ VÀNG

Dữ liệu hợp đồng vàng GC=F được lấy từ Yahoo Finance chart API - GC=F. Khoảng 15/10/2021 - 14/10/2026 có 1257 quan sát; giá gần nhất 4,107.90 USD/oz.

Lợi suất toàn kỳ 132.31%, biến động năm hóa 19.07%, RSI14 31.6. GC=F là tham chiếu quốc tế, không đồng nhất với giá vàng miếng SJC.

Knowledge Graph mô tả lãi suất Fed, USD, lạm phát, địa chính trị và nhu cầu trú ẩn. Những cạnh này là khung giải thích có điều kiện, không phải quan hệ nhân quả cố định.



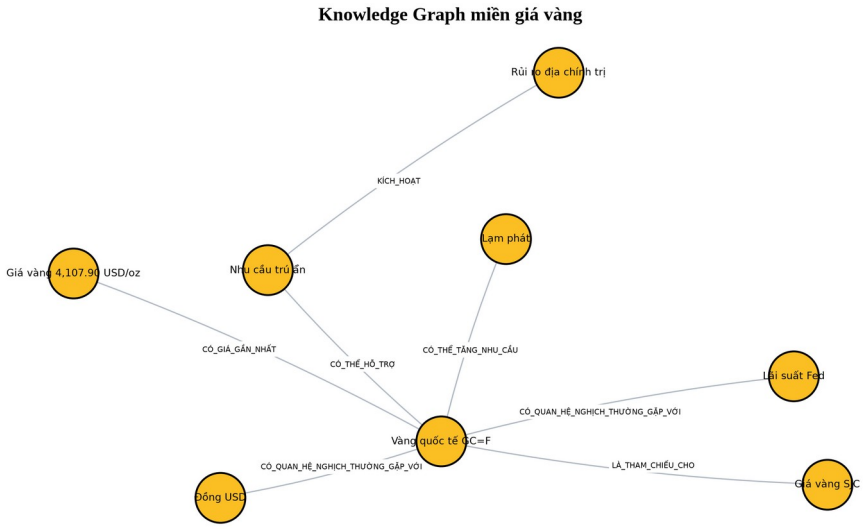
Hình 5.16. Chuỗi giá và chỉ báo vàng

5.6.1. DỮ LIỆU, CHỈ BÁO VÀ KNOWLEDGE GRAPH VÀNG

MA20 hiện là 4,237.78, MA60 là 4,330.82. Chuỗi giữ nguyên lịch phiên của nguồn; không nội suy ngày nghỉ. Biến đích là Close phiên kế tiếp.

Đặc trưng dùng cùng một công thức với FPT để so sánh công bằng: giá và độ trễ, return, MA20, MA60, volatility20, RSI14, thay đổi volume, biên độ ngày và chênh lệch Open-Close.

Chuỗi Rủi ro địa chính trị -> Nhu cầu trú ẩn -> Vàng quốc tế minh họa truy vấn nhiều bước. Quan hệ Vàng quốc tế -> Giá vàng SJC chỉ là tham chiếu, cần thêm tỷ giá và cơ chế thị trường trong nước để giải thích chênh lệch.



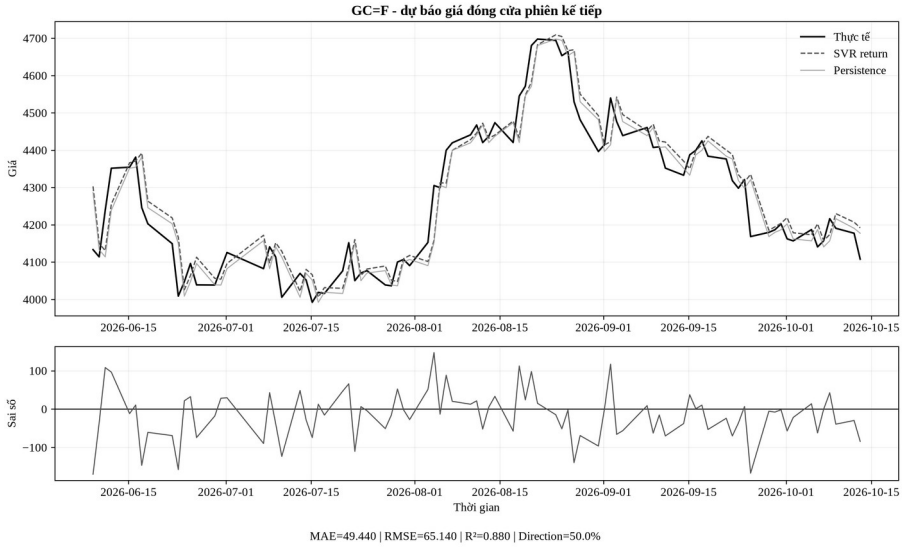
Hình 5.17. Đồ thị tri thức vàng

5.6.2. HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH VÀNG

Vàng dùng cùng quy trình lựa chọn có baseline và mô hình lợi suất như FPT. Không ép chọn mô hình phức tạp nếu Persistence có CV MAE thấp hơn. Train, hiệu chỉnh và test nằm theo đúng thứ tự thời gian, có khoảng cách một phiên ở ranh giới.

Mô hình được chọn: SVR return; CV MAE 38.792; test MAE 49.440; R2 0.880. Baseline MAE 48.267; cải thiện -2.43%.

Test từ 2026-06-09 đến 2026-10-13; bản trước có MAE 50.4701. Mức giảm sai số so với bản trước là 2.04%, nhưng vẫn cần đối chiếu baseline. Khoảng bootstrap khối 5 phiên, 2.000 lần lặp cho mức cải thiện so với baseline: -6.10% đến 2.68%. Khoảng chứa 0 chưa chứng minh cải thiện rõ ràng. GC=F không phải vàng SJC.



Hình 5.18. Mô hình vàng và baseline

5.6.3. HỎI ĐÁP VĨ MÔ VỚI GRAPHRAG

Câu trả lời mẫu: Giá vàng thường nhạy với lãi suất thực, sức mạnh đồng USD, kỳ vọng lạm phát, nhu cầu dự trữ và rủi ro địa chính trị; quan hệ có thể thay đổi theo từng giai đoạn. Chuỗi vàng GC=F gồm 1257 phiên từ 15/10/2021 đến 14/10/2026; giá gần nhất 4,107.90 USD/oz. Quan hệ đồ thị liên quan: Vàng quốc tế GC=F -[CÓ_GIÁ_GẦN_NHẤT]→ Giá vàng 4,107.90 USD/oz; Vàng quốc tế GC=F -[LÀ_THAM_CHIẾU_CHO]→ Giá vàng SJC; Lãi suất Fed -[CÓ_QUAN_HỆ_NGHỊCH_THƯỜNG_GẶP_VỚI]→ Vàng quốc tế GC=F. Đây là phân tích dữ liệu quá khứ, không phải khuyến nghị mua hoặc bán.

Truy xuất văn bản nêu điều kiện và độ bất định; đồ thị cung cấp các cạnh thường gặp và đường đi qua nhu cầu trú ẩn. Hai nguồn bằng chứng cùng xuất hiện giúp câu trả lời tránh tuyệt đối hóa.

Nếu thay bộ sinh mẫu bằng LLM, prompt phải cấm tự tạo mục tiêu giá hoặc xác suất tăng giảm khi evidence không cung cấp. Citation ID và timestamp nên là trường bắt buộc trong JSON đầu ra.

TimeSeriesSplit có gap và mô hình baseline Tieuluan02_06_Nhom2_Giangkt.ipynb

```
train = data.loc[data.index < cal_start].iloc[:-1]
cal = data.loc[(data.index >= cal_start) & (data.index < test_start)].iloc[:-1]
test = data.loc[(data.index >= test_start) & (data.index < "2026-10-14")]
cv = TimeSeriesSplit(n_splits=5, gap=1, test_size=63)
name, search, rows = select_model(candidates, Xtr, ytr, cv, domain)
model = search.best_estimator_
interval = calibrate(model, Xcal, ycal, Xte, yte, relative=True)
pred = model.predict(Xte)
baseline = Xte["Close"].to_numpy()
```

Hình 5.19. Quy trình học máy cho thị trường

5.6.4. GIỚI HẠN VÀ HƯỚNG MỞ RỘNG

Hợp đồng tương lai có ngày đáo hạn và cơ chế nối chuỗi. Spot XAU/USD, futures GC=F và vàng SJC là ba sản phẩm khác nhau; không thể trộn USD/oz với VND/lượng nếu chưa quy đổi.

Mô hình hiện chưa dùng lãi suất thực, DXY, tỷ giá USD/VND hay vị thế ETF. Hướng mở rộng là gắn timestamp cho các yếu tố vĩ mô, thử độ trễ và lưu confidence cùng evidence trên mỗi cạnh.

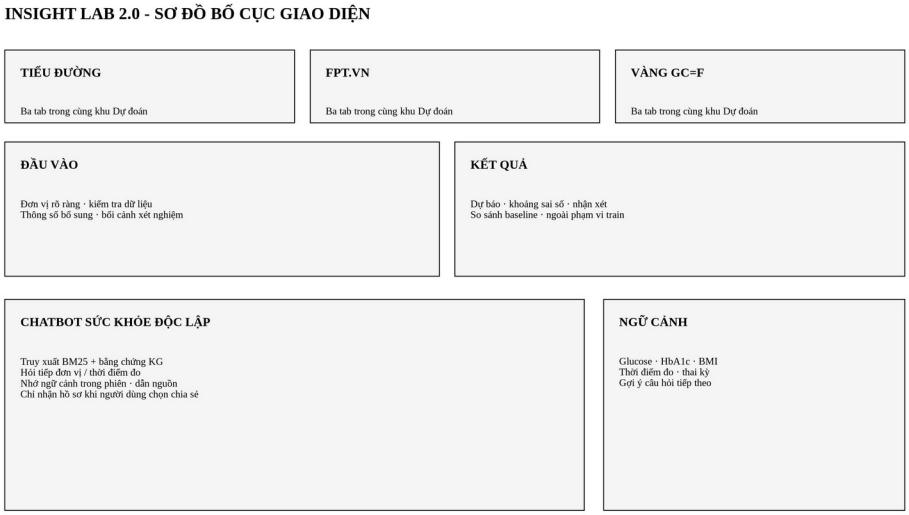
Tương tự chứng khoán, dự báo một bước chỉ minh họa quy trình machine learning có kiểm soát. Nó không phải khuyến nghị mua, bán hay nắm giữ vàng.

5.7. DEPLOYMENT

Insight Lab 2.0 chia thành khu Dự đoán với ba tab Tiểu đường, FPT, Vàng và khu Trợ lý sức khỏe riêng. Người dùng xem kết quả tại từng tab; khi muốn thảo luận, chủ động chọn chia sẻ kết quả sức khỏe cho chatbot.

Web hiển thị biểu đồ lịch sử giá, dự báo điểm, khoảng sai số hiệu chỉnh và so sánh baseline. Đầu vào trống, sai đơn vị hoặc vượt giới hạn được kiểm tra trước suy luận. Giá trị ngoài phạm vi train được đánh dấu để tránh hiểu sai mức tin cậy.

Web: https://rag-httm-chatbot.vercel.app. GitHub: https://github.com/jellalaz/HTTM_ThayQue. Sơ đồ dưới đây minh họa bố cục, không phải ảnh chụp màn hình.



Hình 5.20. Sơ đồ bố cục giao diện gốc

5.7.1. SUY LUẬN MÔ HÌNH TRÊN TRÌNH DUYỆT

Notebook xuất cấu trúc estimator được chọn: scaler, hệ số tuyến tính, vector hỗ trợ và kernel RBF của SVR, cấu trúc cây hoặc công thức Persistence/lợi suất. JavaScript dùng cùng đặc trưng và tham số. Các vector đối chiếu Python/JavaScript được lưu để kiểm tra sai số đóng gói.

Mô hình sức khỏe có sáu trường chính và bốn trường bổ sung. Trường bổ sung trống dùng trung vị tập train và được công bố ở kết quả. HbA1c và bối cảnh đo dùng cho nhận xét xét nghiệm riêng, không bị đưa nhầm vào mô hình chưa học chúng. Điểm tiến triển không được dùng quyết định chẩn đoán hoặc cấp cứu.

Cấu hình thị trường được chọn trên dữ liệu trước test rồi fit lại cho web, chừa 63 phiên gần nhất để hiệu chỉnh khoảng dự báo và bỏ cách một phiên. Chỉ số test lịch sử thuộc

bản fit trước đó, không phải đánh giá ngoài mẫu của bản refit. Web hiển thị riêng hai giai đoạn. Cảnh báo ngoài miền của mô hình lợi suất xét đặc trưng tương đối, không chỉ mức giá tuyệt đối. Web không gửi hồ sơ sức khỏe ra ngoài.

Xuất estimator và web tự chứa Tieuluan02_06_Nhom2_Giangkt.ipynb

```
web_config["models"]["diabetes"] = {
    "estimator": export_estimator(diabetes_model),
    "features": list(X.columns),
    "metrics": diabetes_metrics,
    "interval": diabetes_interval,
}
model_json = json.dumps(web_config, ensure_ascii=False, allow_nan=False)
html = html_template.replace("__MODEL_CONFIG__", model_json)
(WEB_DIR / "index.html").write_text(html, encoding="utf-8")
```

Hình 5.21. Xuất estimator sang JavaScript

5.7.2. CHATBOT DỰ ĐOÁN, TƯ VẤN VÀ HỎI ĐÁP

Chatbot nhận đường huyết, HbA1c, BMI, tuổi, huyết áp tâm thu/tâm trương, huyết áp trung bình, cholesterol toàn phần, LDL, HDL, triglyceride và các đặc trưng nghiên cứu s4/s5. Câu ‘huyết áp 90’ được hỏi lại loại số đo; thiếu đơn vị thì hỏi tiếp. Một tin nhắn có thể chứa nhiều chỉ số, đơn vị được gắn riêng với từng giá trị để tránh lấy nhầm mmol/L của HDL cho glucose.

Các ngưỡng xét nghiệm tham khảo NIDDK/ADA. Đường huyết thấp được xử lý trước bảng phân loại đường cao. Sau bữa ăn thông thường không bị đánh đồng với nghiệm pháp 75 g glucose. Thai kỳ và trẻ em cần bối cảnh riêng. Câu hỏi kê liều thuốc được chuyển thành hướng dẫn trao đổi với bác sĩ.

Kho tri thức mở rộng thành 20 chủ đề, bổ sung nguồn AHA về huyết áp, MedlinePlus về mỡ máu và Mayo Clinic về xét nghiệm cholesterol. Nội dung, đường quan hệ KG và danh mục nguồn được xuất ra health_knowledge.json. BM25 truy xuất kết hợp bộ nhận diện số đo; chưa kết nối LLM ngoài. Khi chia sẻ form, chatbot nhận đủ các trường đã nhập, nhưng loại giá trị trung vị thay thế khỏi nhận xét y khoa. Mã giới tính 1/2 không tự dùng chọn ngưỡng HDL; s5 không tự quy đổi thành xét nghiệm triglyceride. Hồ sơ chỉ giữ trong phiên.

5.8. Đánh giá tổng hợp

Kết quả cho thấy một notebook có thể bao quát tải dữ liệu, tiền xử lý, KG, RAG, đánh giá, báo cáo và web. GraphRAG lai cung cấp thêm loại bằng chứng mà Vector RAG không có: quan hệ có tên và đường đi giữa các thực thể.

Đổi lại, KG yêu cầu thiết kế ontology, kiểm soát quan hệ và cập nhật theo thời gian. Chi phí này chỉ đáng bỏ ra khi bài toán cần giải thích, liên kết đa bước hoặc quản trị tri thức dài hạn.

Bài làm đạt mục tiêu nguyên mẫu tái lập. Hạn chế chính là corpus nhỏ, đánh giá ít câu hỏi, generator dạng mẫu và chưa có chuyên gia miền thẩm định. Những giới hạn này được ghi rõ thay vì suy rộng kết quả.

5.8.1. Kết quả huấn luyện ba miền

Bảng 5.1. Tổng hợp kết quả giữ từ Tiểu luận 02

Miền	Mô hình	CV MAE	Test MAE	R2
diabetes	Ridge	44.443	42.936	0.452
stock	Persistence	1215.974	728.128	0.871
gold	SVR return	38.792	49.440	0.880

Mô hình được chọn theo CV MAE trên tập train. Chỉ số test được tính trên phần dữ liệu giữ lại.

5.8.2. Kết quả truy xuất và bộ ba tri thức

Bảng 5.2. Kết quả smoke test được giữ nguyên

Câu hỏi	Miền	P@3	Graph	Hybrid
Theo dõi gì khi quản lý bệnh tiểu đường?	diabetes	0.67	1	0.92
Mô hình tiểu đường có sai số bao nhiêu?	diabetes	0.33	1	0.58
Giá FPT gần nhất và RSI là bao nhiêu?	stock	0.67	1	0.92
MA20 của cổ phiếu có ý nghĩa gì?	stock	0.33	1	0.58
Giá vàng gần nhất là bao nhiêu?	gold	0.33	1	0.58
Fed, USD và địa chính trị ảnh hưởng vàng ra sao?	gold	0.67	1	0.92

Bảng này là phép kiểm tra hồi quy cho pipeline. Nó không thay thế đánh giá câu trả lời bởi chuyên gia.

Bảng 5.3. Bộ ba tri thức của hệ thống

Subject	Predicate	Object
Bệnh đái tháo đường	ĐƯỢC_THEO_DÕI_BẰNG	Theo dõi HbA1c
Bệnh đái tháo đường	CẦN_TƯ_VẤN_BỞI	Bác sĩ chuyên khoa
Glucose	LIÊN_QUAN_ĐẾN	Bệnh đái tháo đường
BMI	LÀ_YẾU_TỐ_NGUY_CƠ_CỦA	Bệnh đái tháo đường
Huyết áp	CẦN_THEO_DÕI_KHI	Bệnh đái tháo đường
Tuổi	ĐIỀU_CHỈNH_NGUY_CƠ	Bệnh đái tháo đường
Chế độ ăn	HỖ_TRỢ_QUẢN_LÝ	Bệnh đái tháo đường
Vận động	HỖ_TRỢ_QUẢN_LÝ	Bệnh đái tháo đường
FPT.VN	CÓ_GIÁ_GẦN_NHẤT	Giá FPT 57,100 VND
MA20	MÔ_TẢ_XU_HƯỚNG_NGẮN_HẠN_CỦA	FPT.VN
MA60	MÔ_TẢ_XU_HƯỚNG_TRUNG_HẠN_CỦA	FPT.VN
RSI14	ĐO_ĐỘNG_LƯỢNG_CỦA	FPT.VN
Thanh khoản	PHẢN_ÁNH_MỨC_QUAN_TÂM_ĐẾN	FPT.VN
Kết quả kinh doanh	CÓ_THỂ_TÁC_ĐỘNG	FPT.VN
Tâm lý thị trường	CÓ_THỂ_TÁC_ĐỘNG	FPT.VN
Vàng quốc tế GC=F	CÓ_GIÁ_GẦN_NHẤT	Giá vàng 4,107.90 USD/oz
Vàng quốc tế GC=F	LÀ_THAM_CHIẾU_CHO	Giá vàng SJC
Lãi suất Fed	CÓ_QUAN_HỆ_NGHỊCH_THƯỜNG_GẶP_VỚI	Vàng quốc tế GC=F
Đồng USD	CÓ_QUAN_HỆ_NGHỊCH_THƯỜNG_GẶP_VỚI	Vàng quốc tế GC=F
Lạm phát	CÓ_THỂ_TĂNG_NHU_CẦU	Vàng quốc tế GC=F
Rủi ro địa chính trị	KÍCH_HOẠT	Nhu cầu trú ẩn

Subject	Predicate	Object
Nhu cầu trú ẩn	CÓ_THỂ_HỖ_TRỢ	Vàng quốc tế GC=F

5.8.3. Chunking, embedding và giới hạn đánh giá

Embedding là biểu diễn văn bản bằng vector; dense embedding học ngữ nghĩa từ dữ liệu huấn luyện, còn TF-IDF là vector thưa theo tần suất từ. Vector database bổ sung chỉ mục và quản trị truy vấn ở quy mô lớn; prototype hiện lưu ma trận TF-IDF trong bộ nhớ, chưa vận hành một vector database độc lập. Corpus gồm các tài liệu ngắn đã là đơn vị truy xuất, nên không tạo hàng loạt chunk chồng lấn không cần thiết. Với PDF dài, cần chunk theo heading, giữ metadata và thử kích thước/overlap bằng một tập câu hỏi riêng.

Điểm hybrid cũ cộng 0,25 khi có bằng chứng đồ thị là điểm kỹ thuật tự định nghĩa, không phải Accuracy, faithfulness hay xác suất đúng của câu trả lời. Precision theo domain chỉ đo mức đúng miền. Việc mọi câu hỏi tìm được một cạnh không chứng minh cạnh đó đủ để trả lời. Cần phân biệt truy xuất văn bản, độ đúng đường đi và độ bám sát bằng chứng khi đánh giá generator.

5.8.4. Đồng bộ notebook và các chức năng web bổ sung

main.ipynb chứa các bước thực nghiệm mới, pipeline KG/RAG từ Tiểu luận 02, xuất mô hình và đóng gói web. Chế độ full là cấu hình của số liệu trong báo cáo; quick dùng ít dữ liệu/epoch hơn để kiểm tra, không thay số liệu full trong báo cáo. Manifest lưu seed, phiên bản thư viện, hash dataset, checkpoint và cấu hình chạy.

Giao diện Insight Lab giữ khu dự đoán ba miền, trợ lý sức khỏe, màu sắc và bố cục chính. Khu thực nghiệm bổ sung giá nhà, phân loại nhãn nguy cơ tổng hợp, nhận dạng chữ số CNN và dự báo chuỗi. Các mô hình mới sử dụng API /api/predict và trọng số JSON xuất từ notebook. API không gọi dịch vụ LLM, không cần secret và có kiểm tra kích thước cũng như giá trị đầu vào. Chatbot sức khỏe cũ vẫn xử lý trong trình duyệt; dữ liệu nhập vào các demo API được gửi tới máy chủ inference để tính dự đoán.

5.9. Kết luận chương

Điều quan trọng nhất rút ra là không nên coi GraphRAG như một nhãn công nghệ thay thế cho thiết kế dữ liệu. Chất lượng câu trả lời bắt đầu từ việc định nghĩa thực thể, quan hệ, nguồn và phạm vi kết luận. Notebook và web dùng chung artifact để đối chiếu kết quả.